In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')

from matplotlib.lines import Line2D
```

In [2]:
```
df = pd.read_csv('/Users/leonel/Desktop/Respaldo/Desktop/Utel/Redes/Tr
```

In [3]:
```
df.head()
```

Out[3]:

	Transmission_ID	Node_Type	Network_Type	Encryption_Method	Detected_
0	FSDT_00001	Healthcare Node	Hospital Network	Secure Aggregation	Infe
1	FSDT_00002	IoT Sensor Node	Hospital Network	CP-ABE	F
2	FSDT_00003	Healthcare Node	Hospital Network	CP-ABE	Infe
3	FSDT_00004	Cloud Relay	Hospital Network	CP-ABE + MPC	F
4	FSDT_00005	IoT Sensor Node	Remote Health Monitoring	Secure Aggregation	

5 rows × 26 columns

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 13000 entries, 0 to 12999
Data columns (total 26 columns):
 #   Column                                    Non-Null Count  Dtype
---  ------                                    --------------  -----
 0   Transmission_ID                           13000 non-null  str
 1   Node_Type                                 13000 non-null  str
 2   Network_Type                              13000 non-null  str
 3   Encryption_Method                         13000 non-null  str
 4   Detected_Attack_Type                      10343 non-null  str
 5   Privacy_Level                             13000 non-null  str
 6   Number_of_Distributed_Nodes               13000 non-null  int64
 7   Local_Data_Records                        13000 non-null  int64
 8   Bandwidth_Mbps                            13000 non-null  float64
 9   Network_Latency_ms                        13000 non-null  float64
 10  Packet_Loss_Rate                          13000 non-null  float64
 11  Encryption_Time_sec                       13000 non-null  float64
 12  Decryption_Time_sec                       13000 non-null  float64
 13  Secure_Aggregation_Time_sec               13000 non-null  float64
 14  Communication_Overhead_MB                 13000 non-null  float64
 15  Communication_Overhead_Reduction_Percent  13000 non-null  float64
 16  Privacy_Preservation_Score                13000 non-null  float64
 17  Security_Strength_Score                   13000 non-null  float64
 18  Attack_Detection_Rate                     13000 non-null  float64
 19  Trust_Score                               13000 non-null  float64
 20  Transmission_Reliability                  13000 non-null  float64
 21  Model_Accuracy                            13000 non-null  float64
 22  Model_Precision                           13000 non-null  float64
 23  Model_Recall                              13000 non-null  float64
 24  Model_F1_Score                            13000 non-null  float64
 25  Transmission_Status                       13000 non-null  str
dtypes: float64(17), int64(2), str(7)
memory usage: 3.6 MB
```

En la unica variable donde se observan datos nulos es en Detected_Attack_Type, seria viable analizar que tan relevante es esa columna para el analisis a desarrollar y determinar si se rellena con algun valor o se elimana por completo.

In [5]:
```
df.describe()
```

Out[5]:

	Number_of_Distributed_Nodes	Local_Data_Records	Bandwidth_Mbps	N
count	13000.000000	13000.000000	13000.000000	
mean	44.468615	7773.644000	155.177518	
std	23.197243	4172.546869	83.809495	
min	5.000000	602.000000	10.010000	
25%	24.000000	4134.750000	83.077500	
50%	45.000000	7724.000000	154.415000	
75%	65.000000	11422.000000	227.400000	
max	84.000000	14998.000000	299.990000	

Contamos con 19 columnas analizables con describes. y 7 strings.

In [6]:
```
df.columns
```

Out[6]:
```
Index(['Transmission_ID', 'Node_Type', 'Network_Type', 'Encryption_Me
thod',
       'Detected_Attack_Type', 'Privacy_Level', 'Number_of_Distribute
d_Nodes',
       'Local_Data_Records', 'Bandwidth_Mbps', 'Network_Latency_ms',
       'Packet_Loss_Rate', 'Encryption_Time_sec', 'Decryption_Time_se
c',
       'Secure_Aggregation_Time_sec', 'Communication_Overhead_MB',
       'Communication_Overhead_Reduction_Percent',
       'Privacy_Preservation_Score', 'Security_Strength_Score',
       'Attack_Detection_Rate', 'Trust_Score', 'Transmission_Reliabil
ity',
       'Model_Accuracy', 'Model_Precision', 'Model_Recall', 'Model_F1
_Score',
       'Transmission_Status'],
      dtype='str')
```

In [7]:
```
df.Detected_Attack_Type
```

Out[7]:
```
0        Inference Attack
1           Replay Attack
2        Inference Attack
3           Replay Attack
4                     NaN
              ...
12995       Eavesdropping
12996    Inference Attack
12997       Eavesdropping
12998       Eavesdropping
12999       Replay Attack
Name: Detected_Attack_Type, Length: 13000, dtype: str
```

In [8]:
```
df.Detected_Attack_Type.value_counts()
```

Out[8]:
```
Detected_Attack_Type
Model Poisoning     2618
Eavesdropping       2598
Inference Attack    2582
Replay Attack       2545
Name: count, dtype: int64
```

Con esta informacion no es posible determinar si es relevante en el modelo por lo tanto se rellenara las celdas con el valor "Sin Tipo".

In [9]:
```
df.Detected_Attack_Type.nunique()
```

Out[9]:
```
4
```

In [10]:
```
df.Detected_Attack_Type.value_counts()
```

Out[10]:
```
Detected_Attack_Type
Model Poisoning     2618
Eavesdropping       2598
Inference Attack    2582
Replay Attack       2545
Name: count, dtype: int64
```

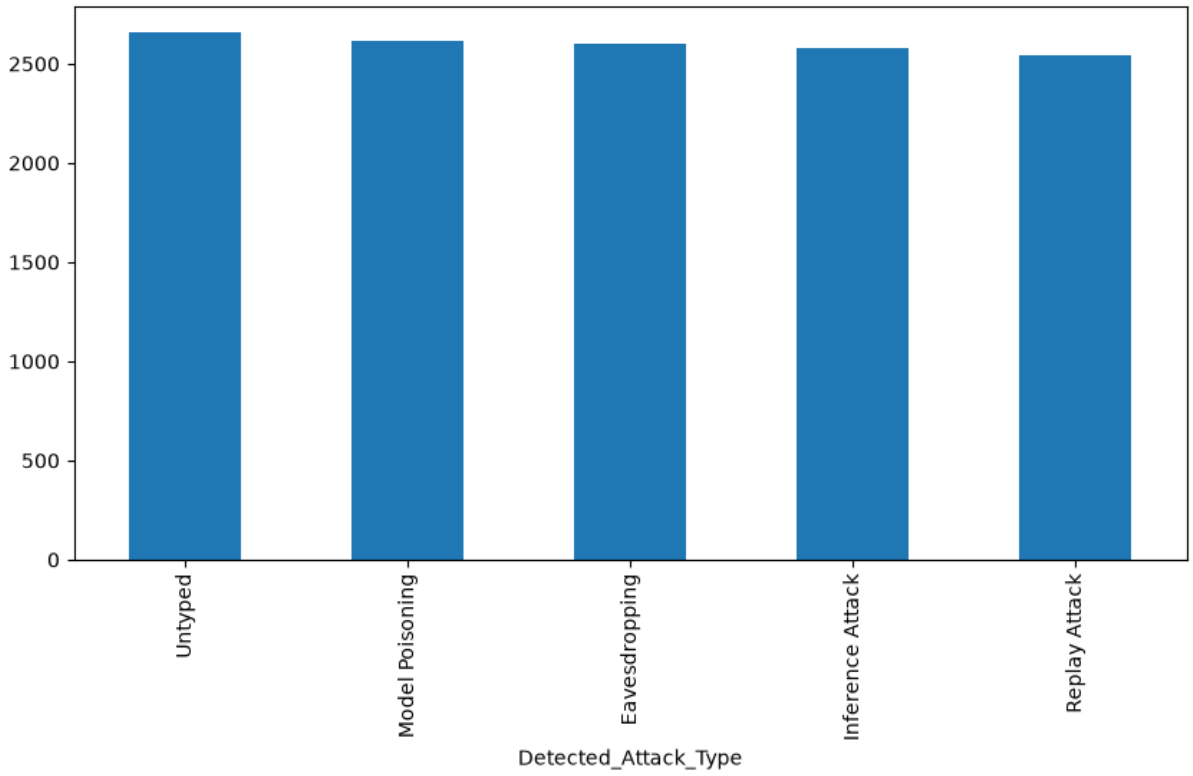
In [11]:
```
df['Detected_Attack_Type'] = df['Detected_Attack_Type'].fillna('Untype
```

In [12]:
```
df['Detected_Attack_Type'].value_counts()
```

Out[12]:
```
Detected_Attack_Type
Untyped             2657
Model Poisoning     2618
Eavesdropping       2598
Inference Attack    2582
Replay Attack       2545
Name: count, dtype: int64
```

In [13]:
```
df['Detected_Attack_Type'].value_counts().plot(kind='bar', figsize=(10
```

Out[13]:
```
<Axes: xlabel='Detected_Attack_Type'>
```



In [14]:
```
df.columns
```

Out[14]:
```
Index(['Transmission_ID', 'Node_Type', 'Network_Type', 'Encryption_Me
thod',
       'Detected_Attack_Type', 'Privacy_Level', 'Number_of_Distribute
d_Nodes',
       'Local_Data_Records', 'Bandwidth_Mbps', 'Network_Latency_ms',
       'Packet_Loss_Rate', 'Encryption_Time_sec', 'Decryption_Time_se
c',
       'Secure_Aggregation_Time_sec', 'Communication_Overhead_MB',
       'Communication_Overhead_Reduction_Percent',
       'Privacy_Preservation_Score', 'Security_Strength_Score',
       'Attack_Detection_Rate', 'Trust_Score', 'Transmission_Reliabil
ity',
       'Model_Accuracy', 'Model_Precision', 'Model_Recall', 'Model_F1
_Score',
       'Transmission_Status'],
      dtype='str')
```

In [15]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 13000 entries, 0 to 12999
Data columns (total 26 columns):
 #   Column                                    Non-Null Count  Dtype
---  ------                                    --------------  -----
 0   Transmission_ID                           13000 non-null  str
 1   Node_Type                                 13000 non-null  str
 2   Network_Type                              13000 non-null  str
 3   Encryption_Method                         13000 non-null  str
 4   Detected_Attack_Type                      13000 non-null  str
 5   Privacy_Level                             13000 non-null  str
 6   Number_of_Distributed_Nodes               13000 non-null  int64
 7   Local_Data_Records                        13000 non-null  int64
 8   Bandwidth_Mbps                            13000 non-null  float64
 9   Network_Latency_ms                        13000 non-null  float64
 10  Packet_Loss_Rate                          13000 non-null  float64
 11  Encryption_Time_sec                       13000 non-null  float64
 12  Decryption_Time_sec                       13000 non-null  float64
 13  Secure_Aggregation_Time_sec               13000 non-null  float64
 14  Communication_Overhead_MB                 13000 non-null  float64
 15  Communication_Overhead_Reduction_Percent  13000 non-null  float64
 16  Privacy_Preservation_Score                13000 non-null  float64
 17  Security_Strength_Score                   13000 non-null  float64
 18  Attack_Detection_Rate                     13000 non-null  float64
 19  Trust_Score                               13000 non-null  float64
 20  Transmission_Reliability                  13000 non-null  float64
 21  Model_Accuracy                            13000 non-null  float64
 22  Model_Precision                           13000 non-null  float64
 23  Model_Recall                              13000 non-null  float64
 24  Model_F1_Score                            13000 non-null  float64
 25  Transmission_Status                       13000 non-null  str
dtypes: float64(17), int64(2), str(7)
memory usage: 3.6 MB
```

Contamos con 26 columnas de las cuales ya no tenemos valores no nulos, todas tienen 13,000 celdas completas, son 6 variables de texto, por lo que se prevee posible hacer numericas por medio de algun metodo como one hot encoding, label o ordinal encoding, ademas de la variable objetivo con sus metricas de evaluacion por lo que

In [16]:
```
df.shape
```

Out[16]:
```
(13000, 26)
```

In [17]:
```
df.columns
```

Out[17]:
```
Index(['Transmission_ID', 'Node_Type', 'Network_Type', 'Encryption_Me
thod',
       'Detected_Attack_Type', 'Privacy_Level', 'Number_of_Distribute
d_Nodes',
       'Local_Data_Records', 'Bandwidth_Mbps', 'Network_Latency_ms',
       'Packet_Loss_Rate', 'Encryption_Time_sec', 'Decryption_Time_se
c',
       'Secure_Aggregation_Time_sec', 'Communication_Overhead_MB',
       'Communication_Overhead_Reduction_Percent',
       'Privacy_Preservation_Score', 'Security_Strength_Score',
       'Attack_Detection_Rate', 'Trust_Score', 'Transmission_Reliabil
ity',
       'Model_Accuracy', 'Model_Precision', 'Model_Recall', 'Model_F1
_Score',
       'Transmission_Status'],
      dtype='str')
```

In [18]:
```
df.describe()
```

Out[18]:

	Number_of_Distributed_Nodes	Local_Data_Records	Bandwidth_Mbps	N
count	13000.000000	13000.000000	13000.000000	
mean	44.468615	7773.644000	155.177518	
std	23.197243	4172.546869	83.809495	
min	5.000000	602.000000	10.010000	
25%	24.000000	4134.750000	83.077500	
50%	45.000000	7724.000000	154.415000	
75%	65.000000	11422.000000	227.400000	
max	84.000000	14998.000000	299.990000	

In [19]:
```
df.describe().T
```

Out[19]:

	count	mean	st
Number_of_Distributed_Nodes	13000.0	44.468615	23.19724
Local_Data_Records	13000.0	7773.644000	4172.54686
Bandwidth_Mbps	13000.0	155.177518	83.80949
Network_Latency_ms	13000.0	102.810888	56.27953
Packet_Loss_Rate	13000.0	3.018099	1.73524
Encryption_Time_sec	13000.0	1.492757	0.81069
Decryption_Time_sec	13000.0	1.335168	0.72795
Secure_Aggregation_Time_sec	13000.0	3.657381	1.93629
Communication_Overhead_MB	13000.0	39.897935	17.30111
Communication_Overhead_Reduction_Percent	13000.0	31.072245	6.29583
Privacy_Preservation_Score	13000.0	84.632861	8.30751
Security_Strength_Score	13000.0	85.579621	7.80452
Attack_Detection_Rate	13000.0	88.450564	6.00524
Trust_Score	13000.0	82.070174	9.81066
Transmission_Reliability	13000.0	84.450197	8.42198
Model_Accuracy	13000.0	94.496008	2.59874
Model_Precision	13000.0	94.039822	2.88291
Model_Recall	13000.0	93.504028	3.16679
Model_F1_Score	13000.0	93.721655	2.13719

In [20]:
```
df.describe(include='object').T
```

Out[20]:

	count	unique	top	freq
Transmission_ID	13000	13000	FSDT_00001	1
Node_Type	13000	5	Cloud Relay	2640
Network_Type	13000	4	Remote Health Monitoring	3374
Encryption_Method	13000	4	CP-ABE	3307
Detected_Attack_Type	13000	5	Untyped	2657
Privacy_Level	13000	4	Critical	3388
Transmission_Status	13000	4	Secure	4794

In [21]:
```
df.columns.tolist()
```

Out[21]:
```
['Transmission_ID',
 'Node_Type',
 'Network_Type',
 'Encryption_Method',
 'Detected_Attack_Type',
 'Privacy_Level',
 'Number_of_Distributed_Nodes',
 'Local_Data_Records',
 'Bandwidth_Mbps',
 'Network_Latency_ms',
 'Packet_Loss_Rate',
 'Encryption_Time_sec',
 'Decryption_Time_sec',
 'Secure_Aggregation_Time_sec',
 'Communication_Overhead_MB',
 'Communication_Overhead_Reduction_Percent',
 'Privacy_Preservation_Score',
 'Security_Strength_Score',
 'Attack_Detection_Rate',
 'Trust_Score',
 'Transmission_Reliability',
 'Model_Accuracy',
 'Model_Precision',
 'Model_Recall',
 'Model_F1_Score',
 'Transmission_Status']
```

In [22]:
```
df['Detected_Attack_Type'].value_counts()
```

Out[22]:
```
Detected_Attack_Type
Untyped             2657
Model Poisoning     2618
Eavesdropping       2598
Inference Attack    2582
Replay Attack       2545
Name: count, dtype: int64
```

In [23]:
```
columnas_cat = df.select_dtypes(include='object').columns.to_list()

print(f"Se encontraron {len(columnas_cat)} columnas categóricas: {colu

for col in columnas_cat:
    print(f"Conteo de la columna: '{col}'")
    print(df[col].value_counts())
    print("\n")
```

```
Se encontraron 7 columnas categóricas: ['Transmission_ID', 'Node_Type',
'Network_Type', 'Encryption_Method', 'Detected_Attack_Type', 'Privacy_L
evel', 'Transmission_Status']
Conteo de la columna: 'Transmission_ID'
Transmission_ID
```

```
FSDT_00001    1
FSDT_00002    1
FSDT_00003    1
FSDT_00004    1
FSDT_00005    1
             ..
FSDT_12996    1
FSDT_12997    1
FSDT_12998    1
FSDT_12999    1
FSDT_13000    1
Name: count, Length: 13000, dtype: int64


Conteo de la columna: 'Node_Type'
Node_Type
Cloud Relay        2640
IoT Sensor Node    2609
Mobile Device      2598
Edge Gateway       2577
Healthcare Node    2576
Name: count, dtype: int64


Conteo de la columna: 'Network_Type'
Network_Type
Remote Health Monitoring    3374
Hospital Network            3265
Industrial IoT              3218
Smart City IoT              3143
Name: count, dtype: int64


Conteo de la columna: 'Encryption_Method'
Encryption_Method
CP-ABE                3307
CP-ABE + MPC          3254
Secure Aggregation    3237
Hybrid CP-ABE         3202
Name: count, dtype: int64


Conteo de la columna: 'Detected_Attack_Type'
Detected_Attack_Type
Untyped             2657
Model Poisoning     2618
Eavesdropping       2598
Inference Attack    2582
Replay Attack       2545
Name: count, dtype: int64
```

```
Conteo de la columna: 'Privacy_Level'
Privacy_Level
Critical    3388
Medium      3244
High        3203
Low         3165
Name: count, dtype: int64


Conteo de la columna: 'Transmission_Status'
Transmission_Status
Secure               4794
Moderately Secure    4284
Highly Secure        2499
At Risk              1423
Name: count, dtype: int64
```

In [24]:
```
df.head()
```

Out[24]:

	Transmission_ID	Node_Type	Network_Type	Encryption_Method	Detected_
0	FSDT_00001	Healthcare Node	Hospital Network	Secure Aggregation	Infe
1	FSDT_00002	IoT Sensor Node	Hospital Network	CP-ABE	F
2	FSDT_00003	Healthcare Node	Hospital Network	CP-ABE	Infe
3	FSDT_00004	Cloud Relay	Hospital Network	CP-ABE + MPC	F
4	FSDT_00005	IoT Sensor Node	Remote Health Monitoring	Secure Aggregation	

5 rows × 26 columns

Vamos a eliminar columnas que se usaron en el analisis original del DF como f1-score, accuracy, etc. Para evitar sesgos o fuga de datos, se dejara el analisis de riesgo que se plantea podria ser funcional tambien

Se plantea utilizar Isolation Forest como modelo principal para determinar lo que es normal y anormal sin sesgos humanos, solo metodo no supervisado

Pasos a seguir:

1. Eliminar columnas que generan sesgo
2. Generar heatmap inicial sin considerar variables categoricas

3. Descriptivos Univariables, graficas, etc.
4. Generar variables categoricas a numericas con metodos de transformacion
5. Analisis bivariables y multivariable
6. Desarrollar modelo o modelos de analisis (60% entrenamiento, 30% evaluacion, 10% validacion)
7. Conclusiones

In [25]:
```
df_limpio = df.drop(columns=['Model_Accuracy', 'Model_F1_Score', 'Mode
```

In [26]:
```
df_limpio.head()
```

Out[26]:

	Transmission_ID	Node_Type	Network_Type	Encryption_Method	Detected_
0	FSDT_00001	Healthcare Node	Hospital Network	Secure Aggregation	Infe
1	FSDT_00002	IoT Sensor Node	Hospital Network	CP-ABE	F
2	FSDT_00003	Healthcare Node	Hospital Network	CP-ABE	Infe
3	FSDT_00004	Cloud Relay	Hospital Network	CP-ABE + MPC	F
4	FSDT_00005	IoT Sensor Node	Remote Health Monitoring	Secure Aggregation	

5 rows × 22 columns

In [27]:
```
df_numericas = df_limpio.select_dtypes(include=np.number)
df_categoricas = df_limpio.select_dtypes(include=['object']).drop(colu
```

In [28]:
```
df_numericas.head(5)
```

Out[28]:

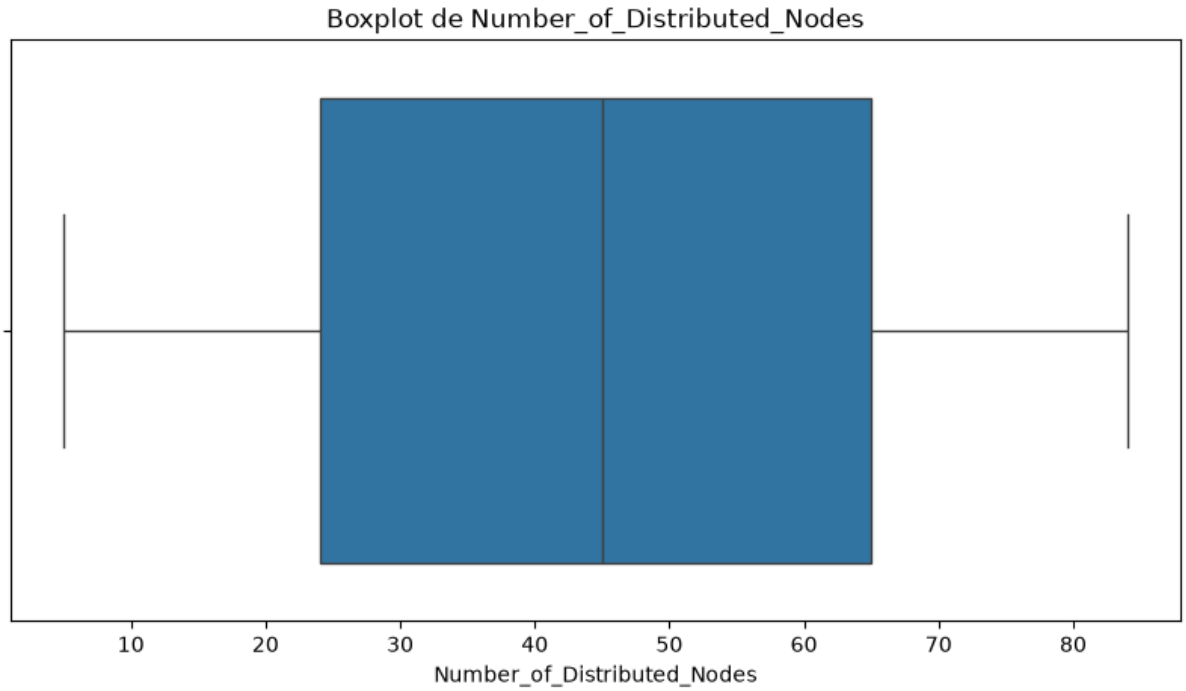
	Number_of_Distributed_Nodes	Local_Data_Records	Bandwidth_Mbps	Netwo
0	56	1460	63.20	
1	53	9266	187.44	
2	13	8113	38.32	
3	66	1595	183.39	
4	5	2627	95.11	

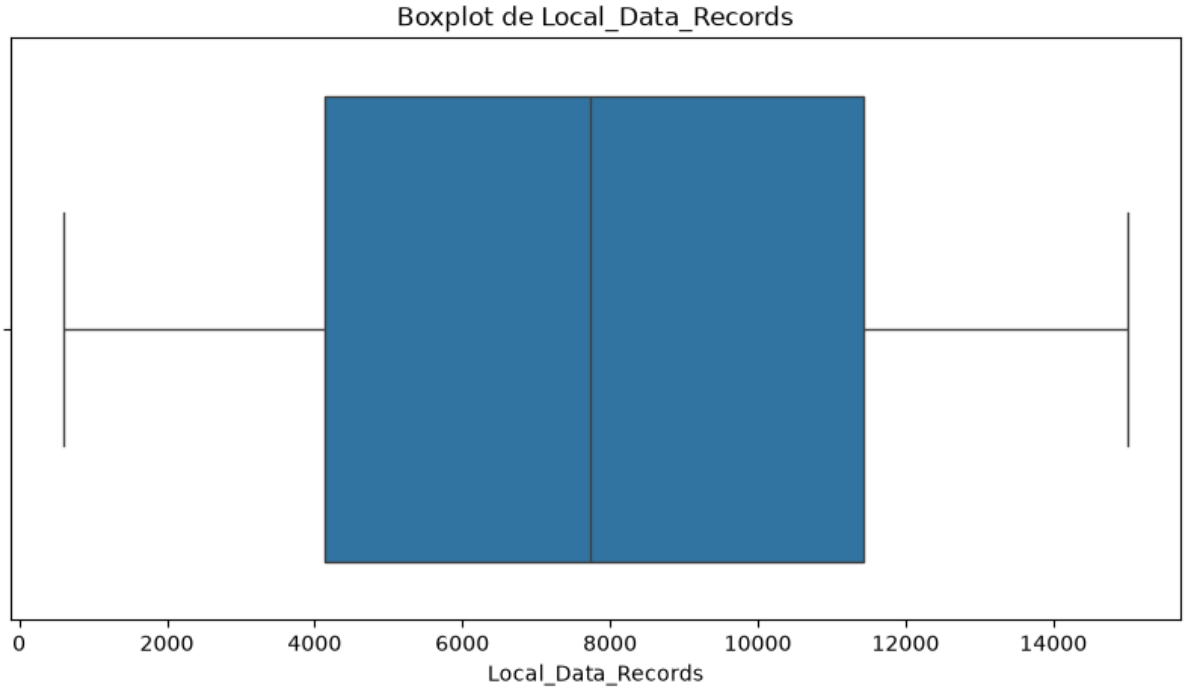
In [29]:
```
df_categoricas.head(5)
```

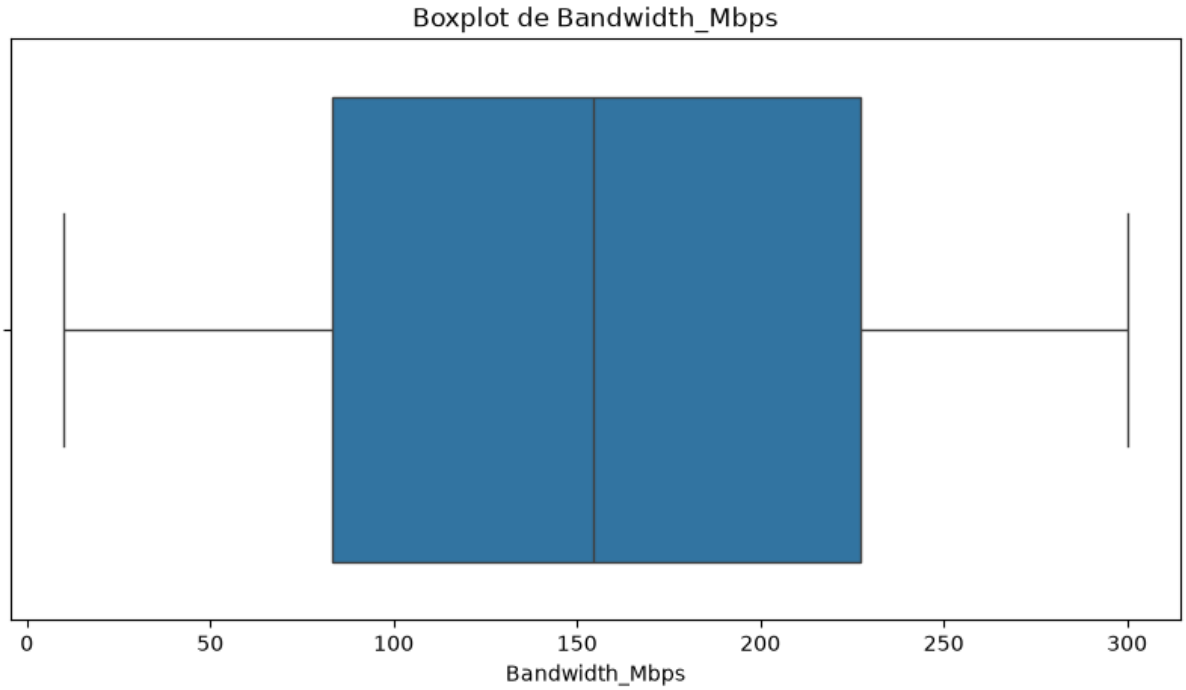
Out[29]:

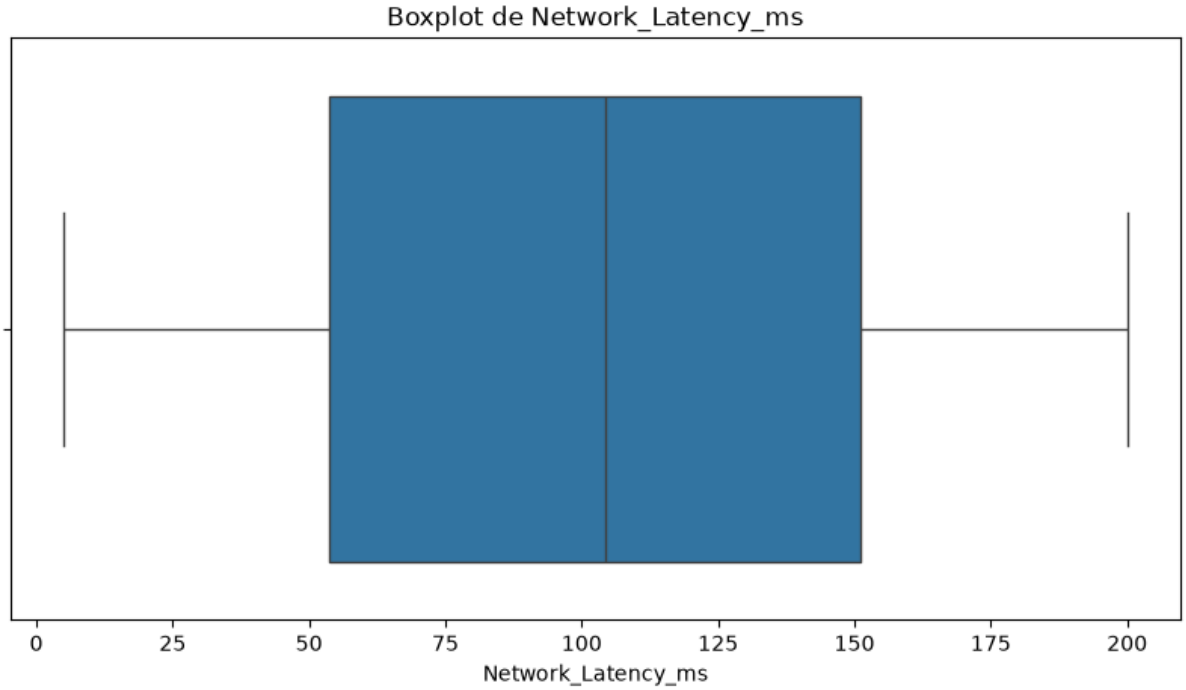
	Node_Type	Network_Type	Encryption_Method	Detected_Attack_Type	Priva
0	Healthcare Node	Hospital Network	Secure Aggregation	Inference Attack	
1	IoT Sensor Node	Hospital Network	CP-ABE	Replay Attack	
2	Healthcare Node	Hospital Network	CP-ABE	Inference Attack	
3	Cloud Relay	Hospital Network	CP-ABE + MPC	Replay Attack	
4	IoT Sensor Node	Remote Health Monitoring	Secure Aggregation	Untyped	

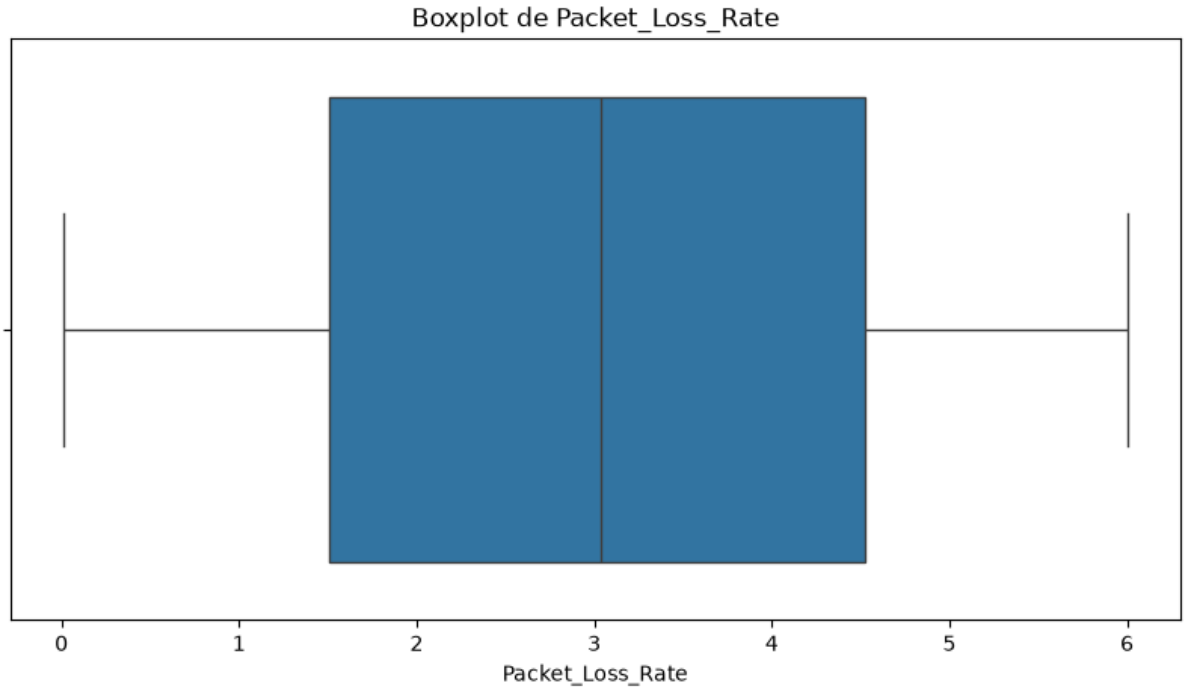
In [30]:
```
for col in df_numericas.columns:
    plt.figure(figsize=(10, 5))
    sns.boxplot(x=df_numericas[col])
    plt.title(f'Boxplot de {col}')
    plt.show()
```

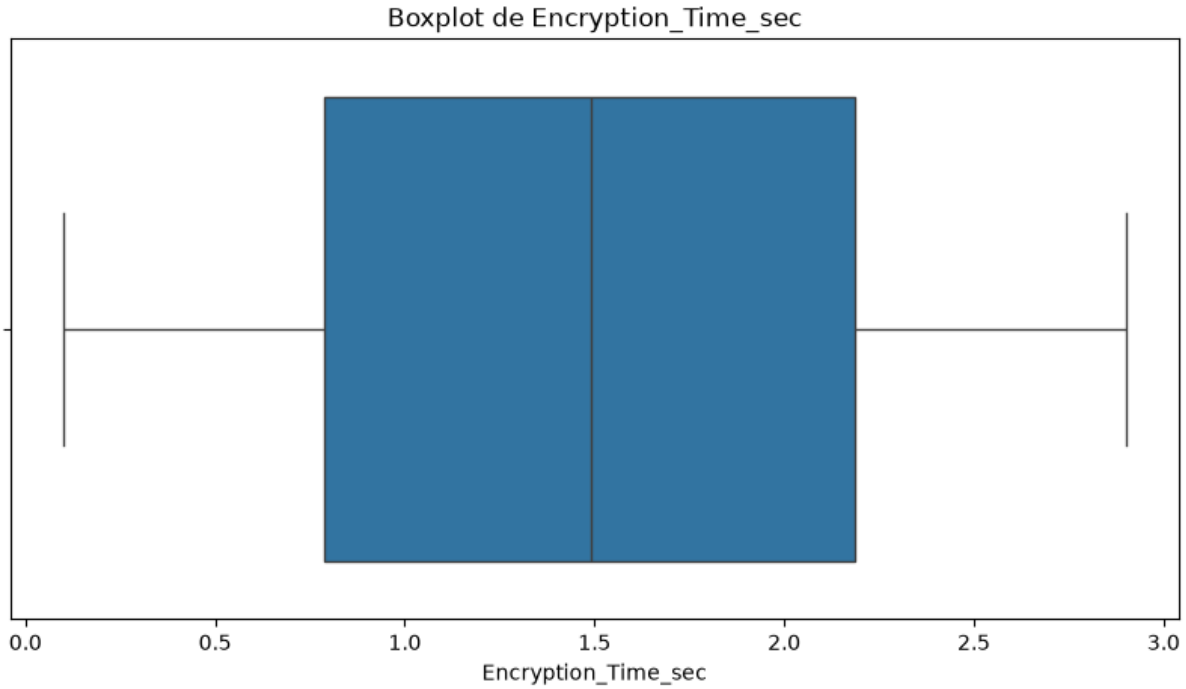


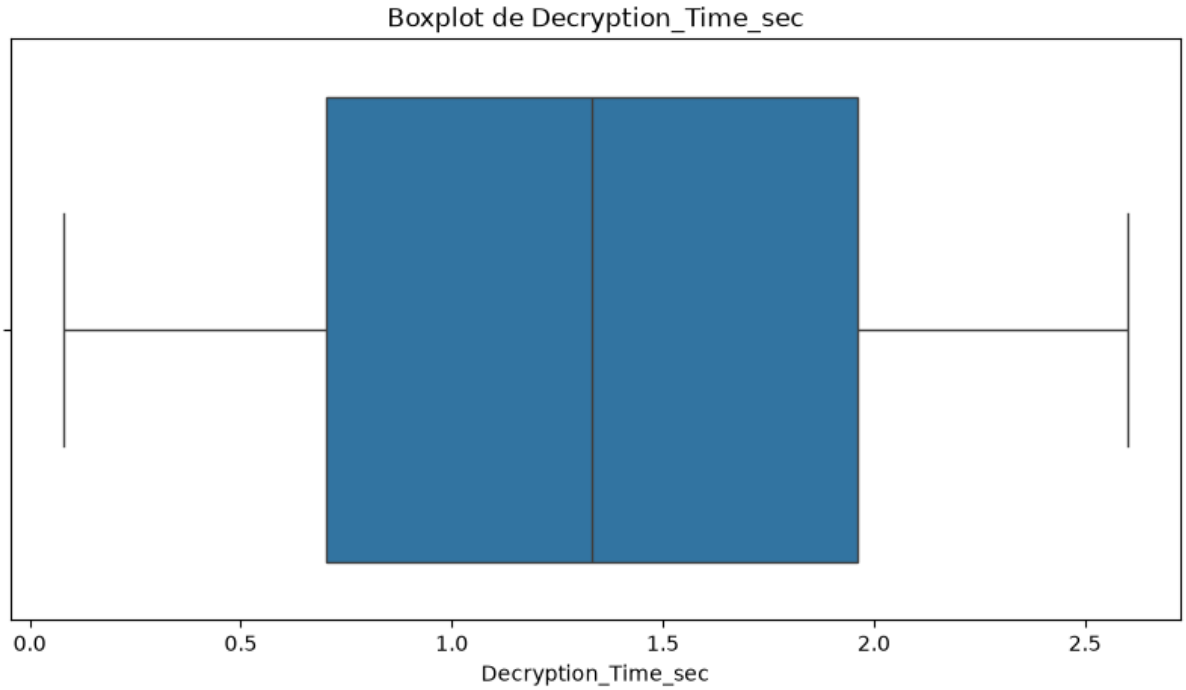

Boxplot de Local_Data_Records
0
2000
4000
6000
8000
10000
12000
14000
Local_Data_Records

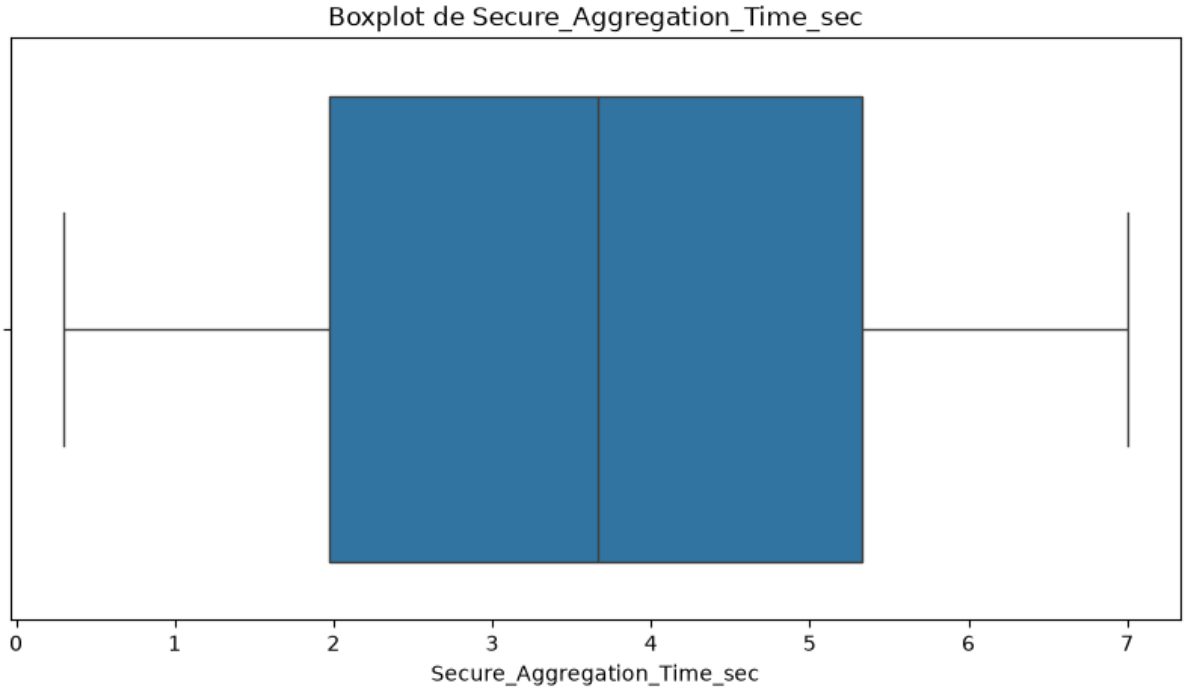

Boxplot de Bandwidth_Mbps
0
50
100
150
200
250
300
Bandwidth_Mbps

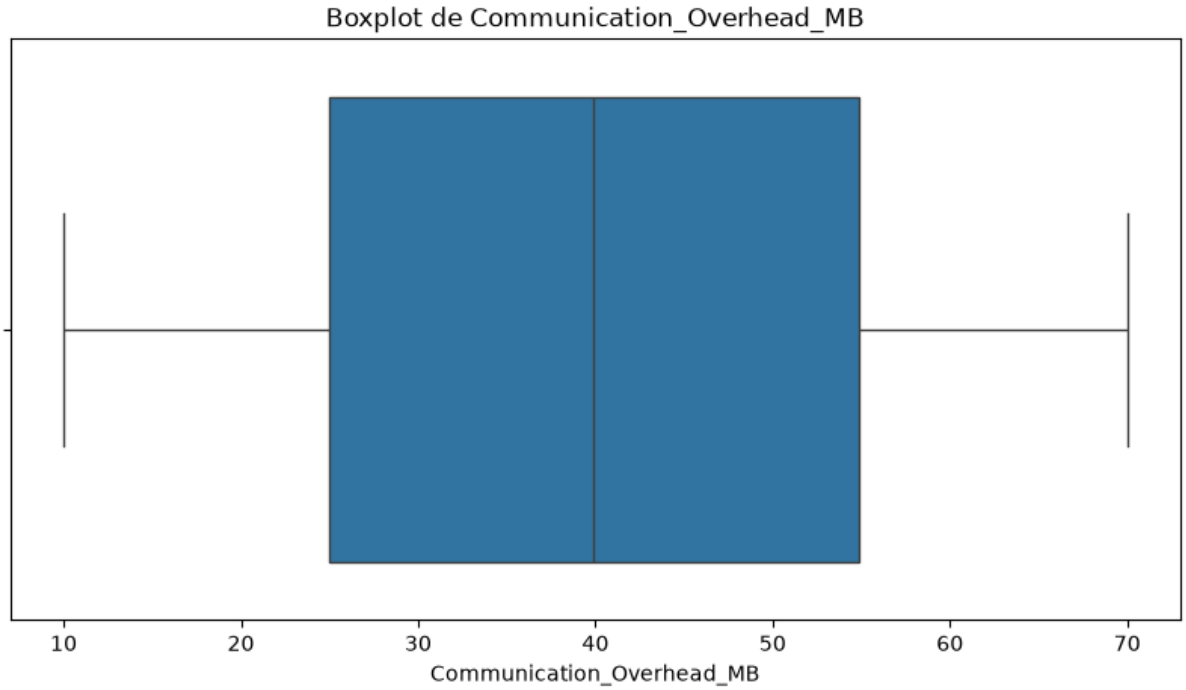

Boxplot de Network_Latency_ms
0
25
50
75
100
125
150
175
200
Network_Latency_ms

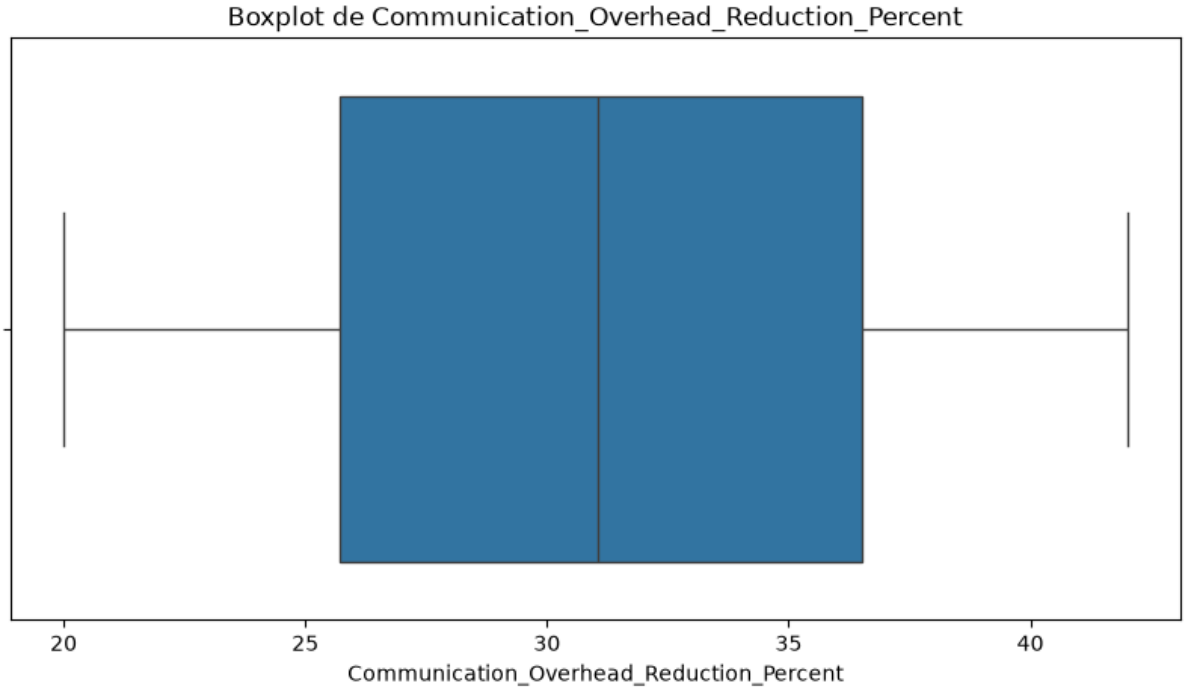

Boxplot de Packet_Loss_Rate
0
1
2
3
4
5
6
Packet_Loss_Rate

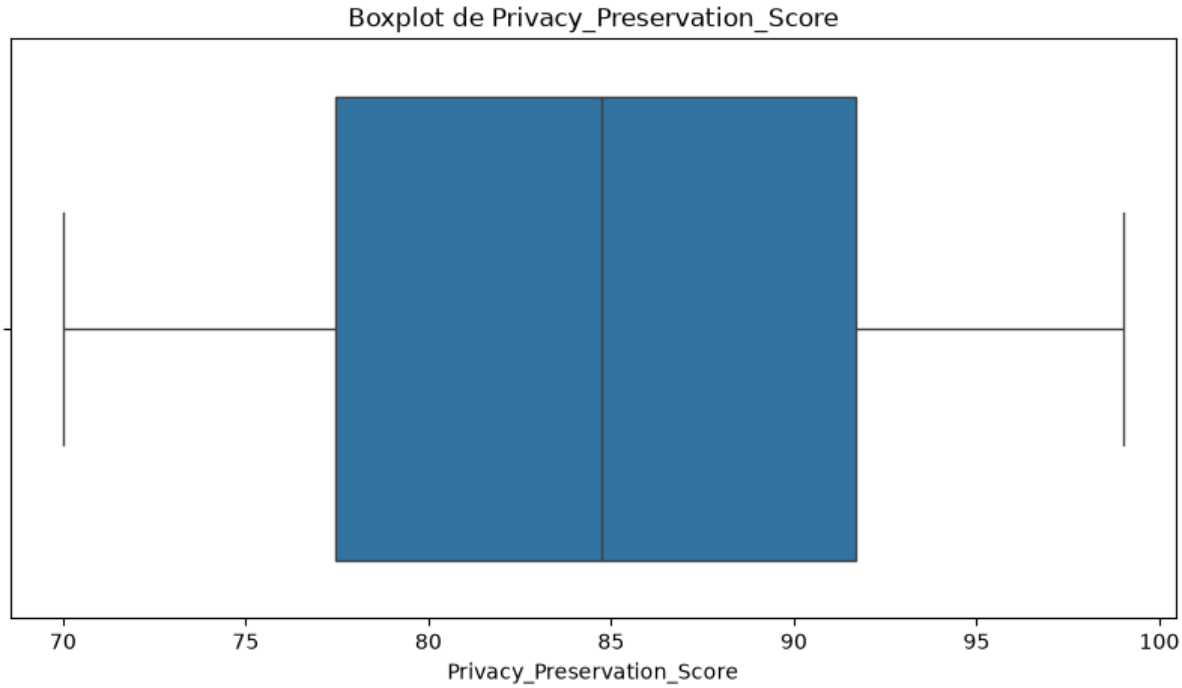

Boxplot de Encryption_Time_sec
0.0
0.5
1.0
1.5
2.0
2.5
3.0
Encryption_Time_sec

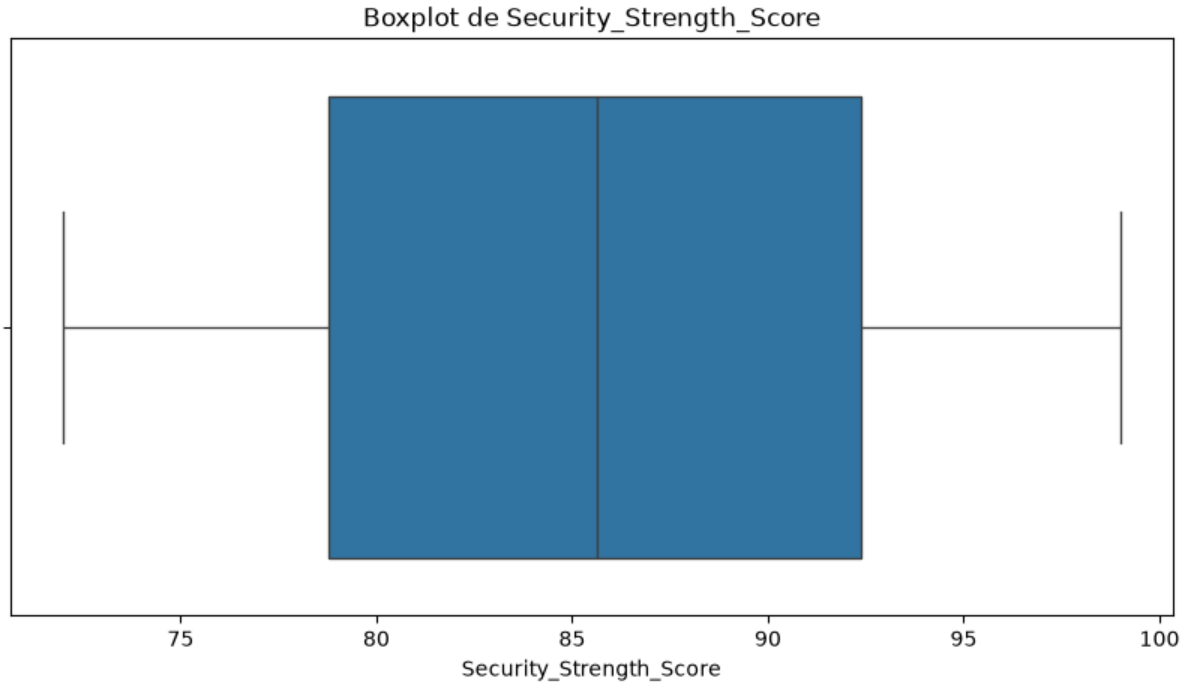

Boxplot de Decryption_Time_sec
0.0
0.5
1.0
1.5
2.0
2.5
Decryption_Time_sec

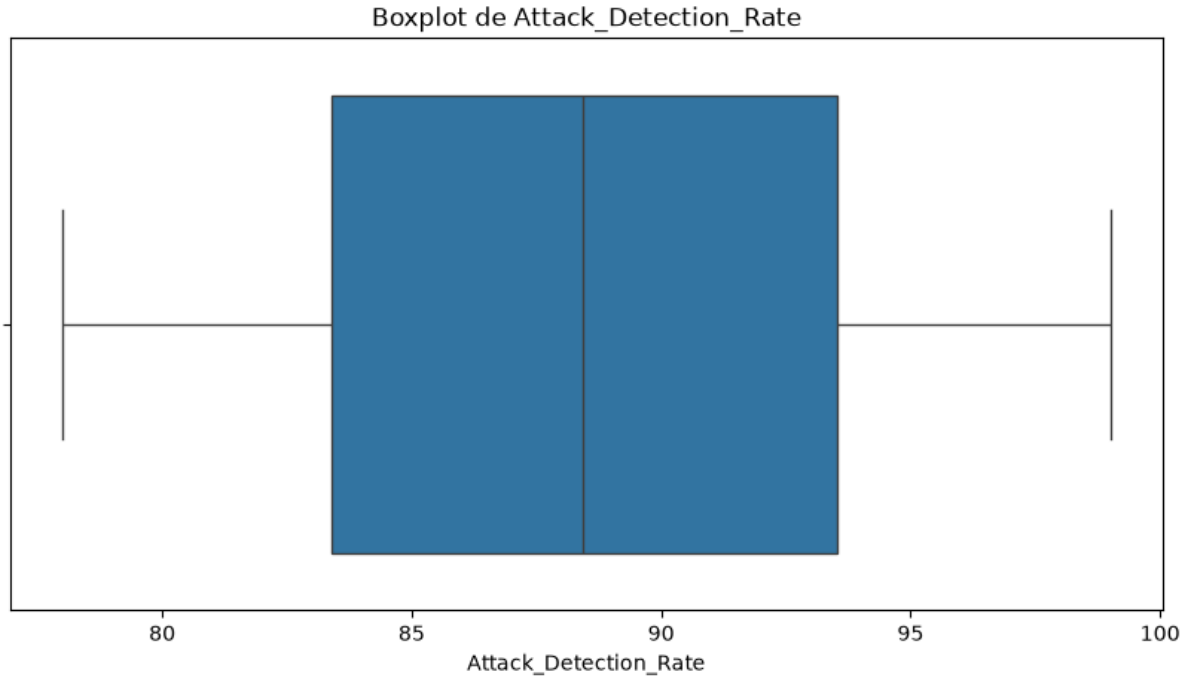

Boxplot de Secure_Aggregation_Time_sec
0
1
2
3
4
5
6
7
Secure_Aggregation_Time_sec

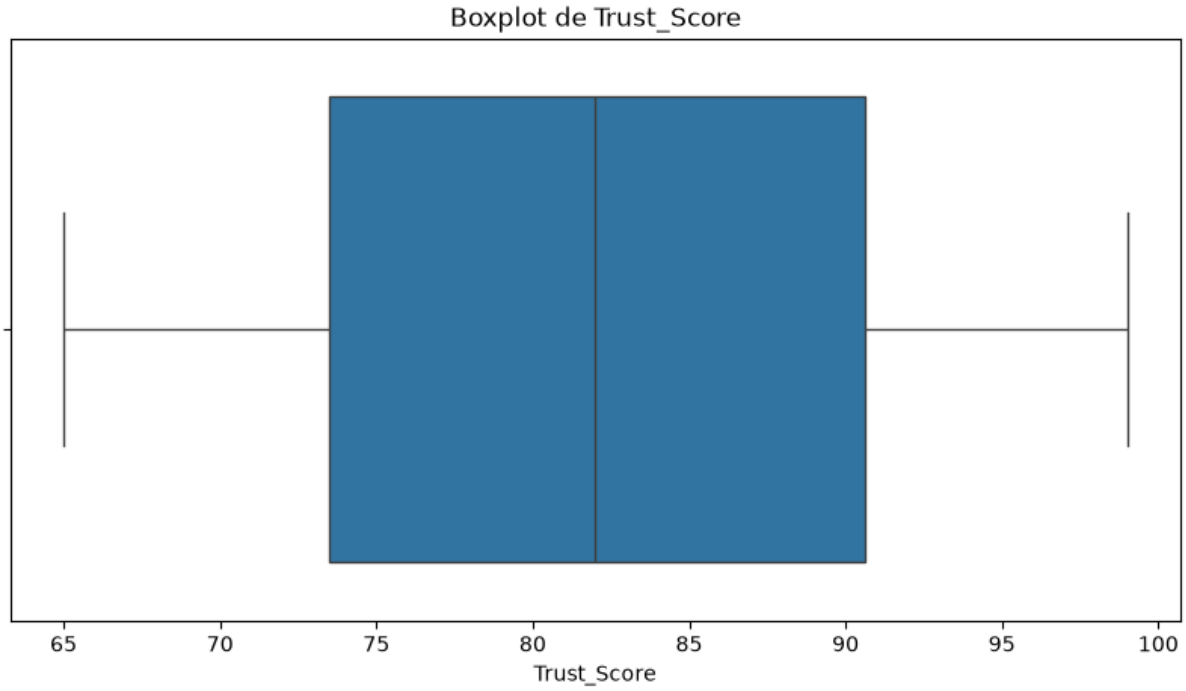

Boxplot de Communication_Overhead_MB
10
20
30
40
50
60
70
Communication_Overhead_MB

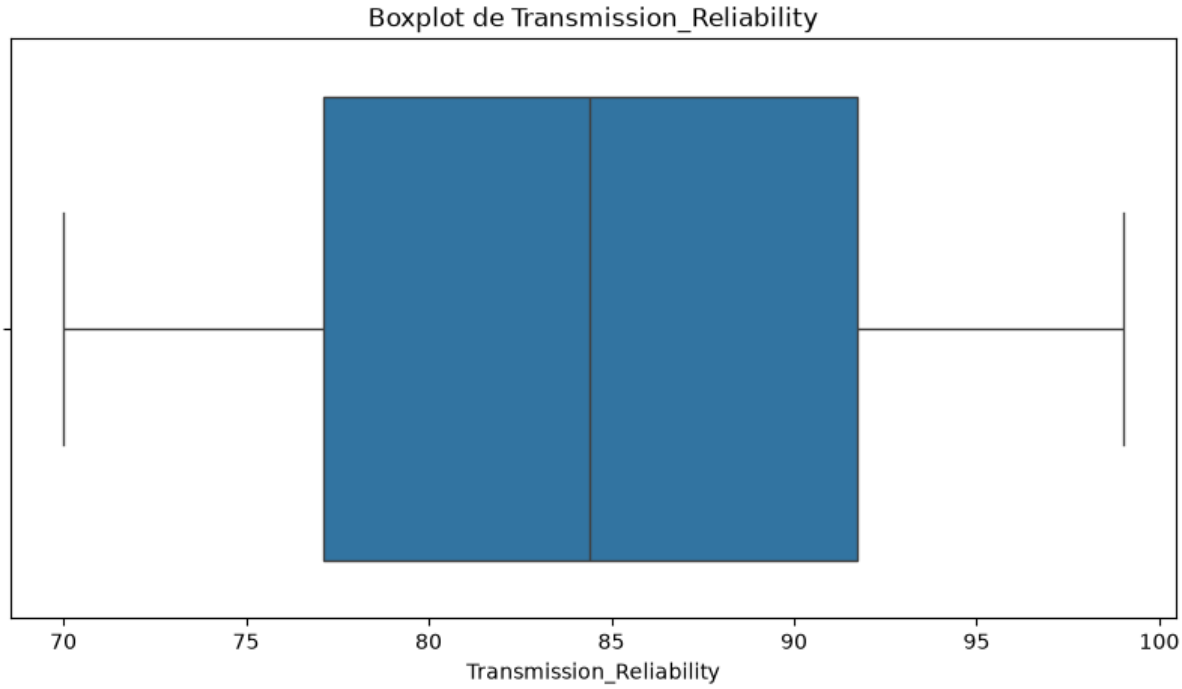

Boxplot de Communication_Overhead_Reduction_Percent
20
25
30
35
40
Communication_Overhead_Reduction_Percent


Boxplot de Privacy_Preservation_Score
70
75
80
85
90
95
100
Privacy_Preservation_Score

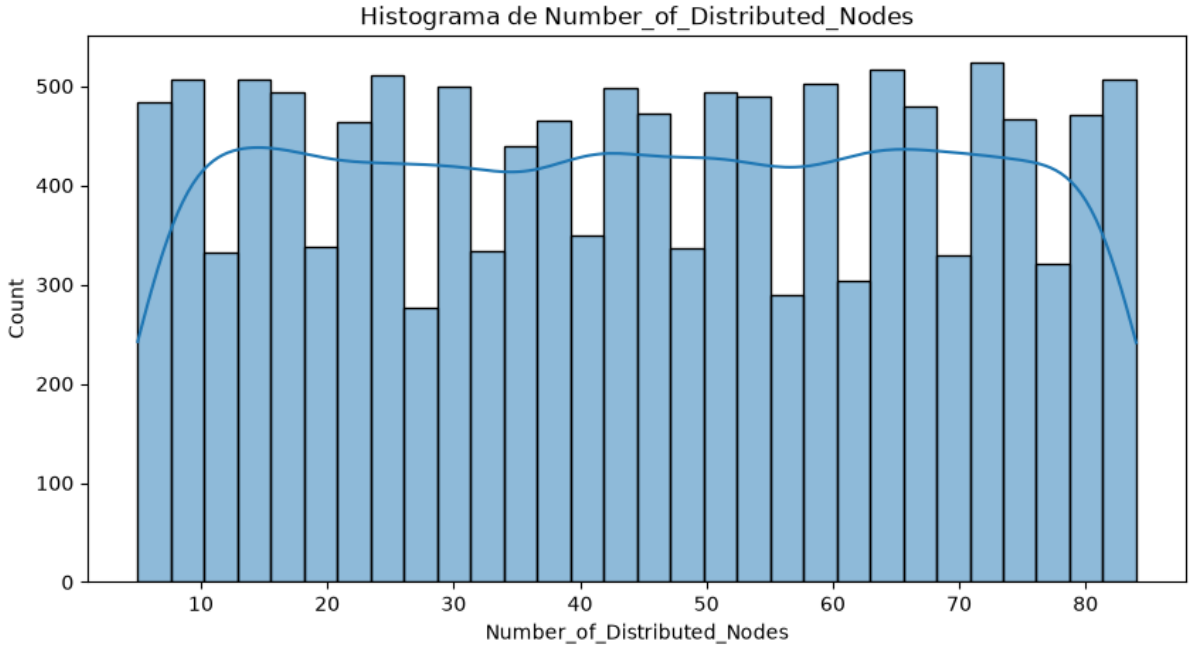

Boxplot de Security_Strength_Score
75
80
85
90
95
100
Security_Strength_Score

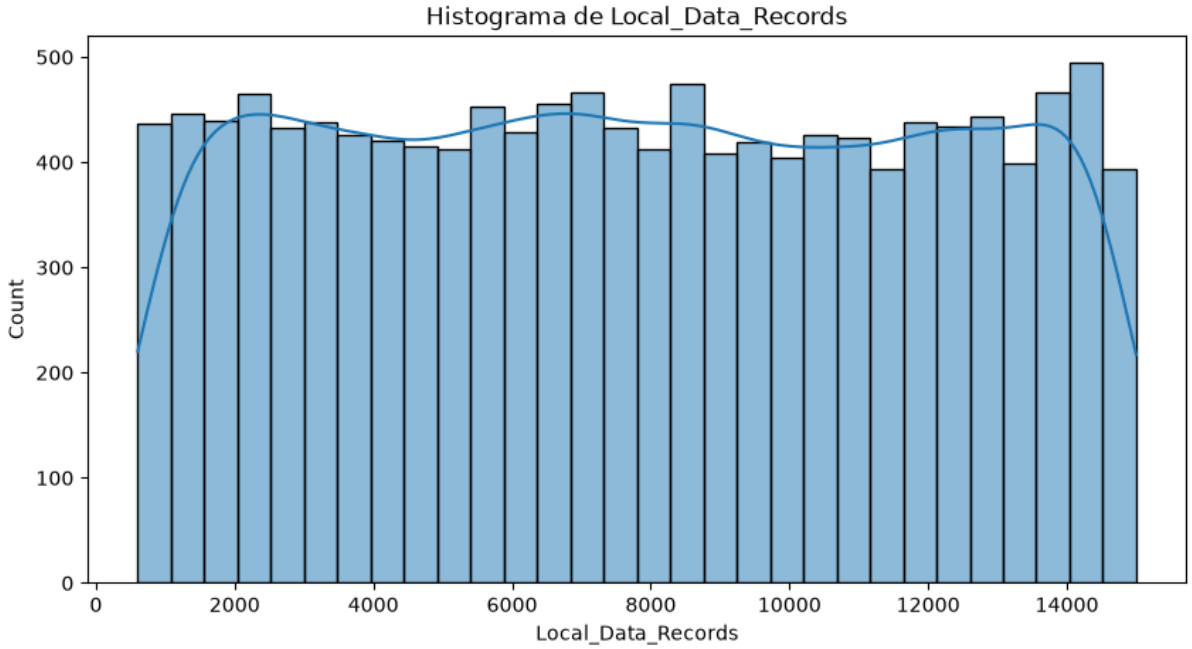

Boxplot de Attack_Detection_Rate
80
85
90
95
100
Attack_Detection_Rate

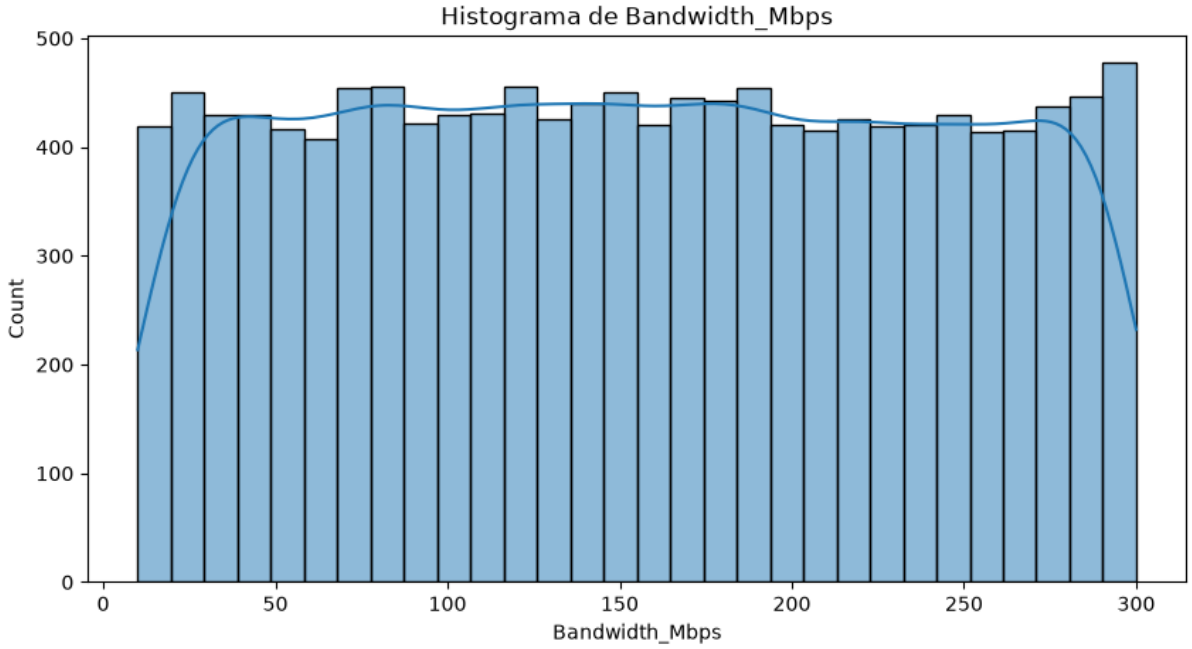


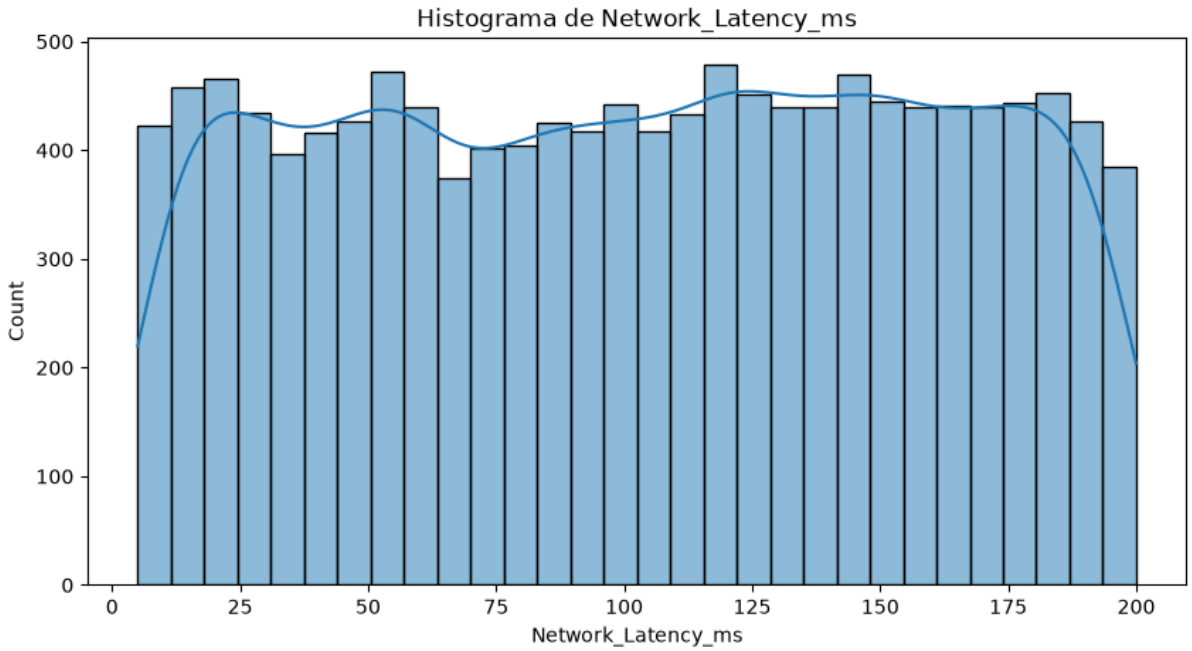


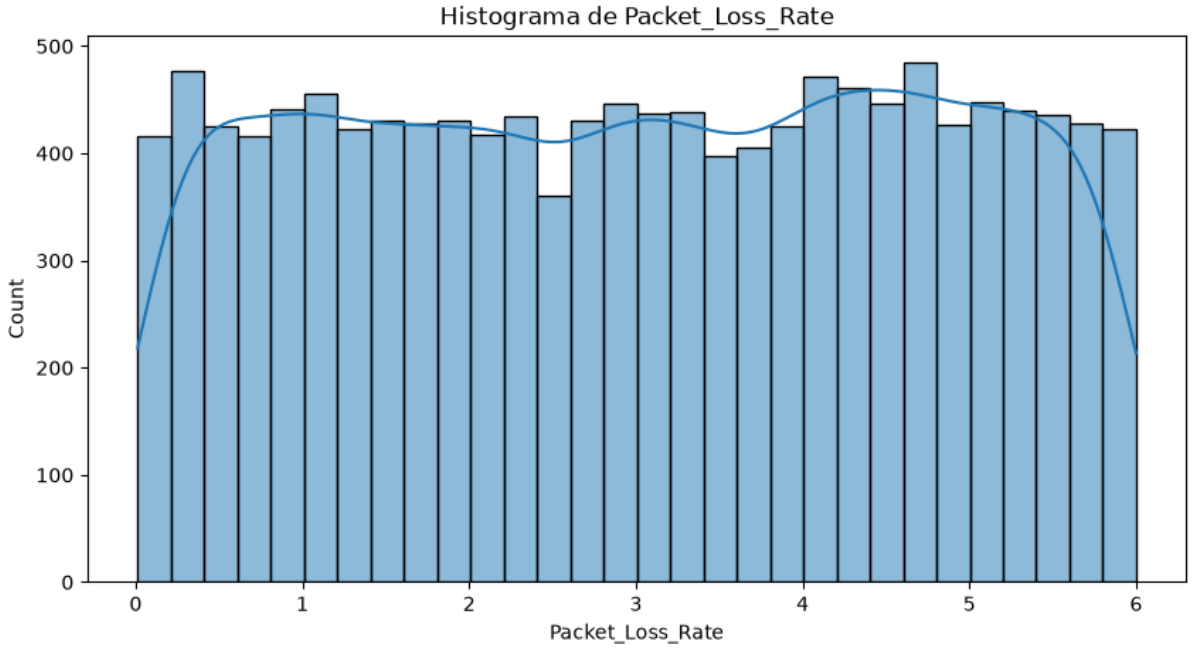
In [31]:
```
for col in df_numericas.columns:
    plt.figure(figsize=(10, 5))
    sns.histplot(x=df_numericas[col], bins=30, kde=True)
    plt.title(f'Histograma de {col}')
    plt.show()
```

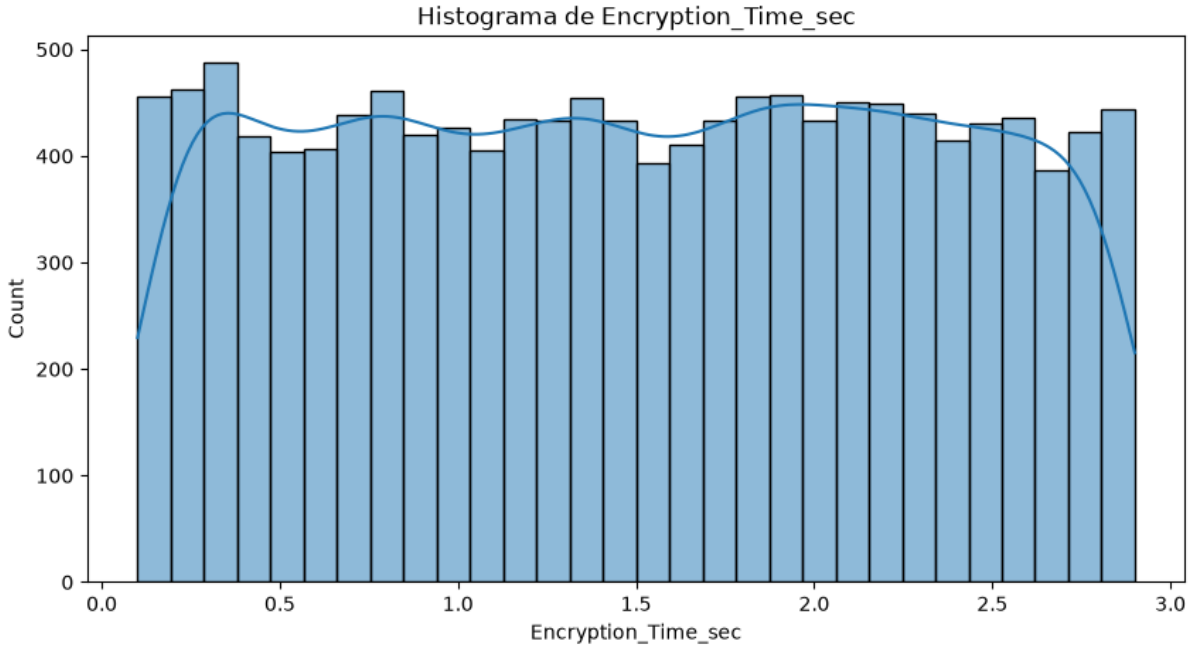

Histograma de Number_of_Distributed_Nodes
Count
0
100
200
300
400
500
10
20
30
40
50
60
70
80
Number_of_Distributed_Nodes

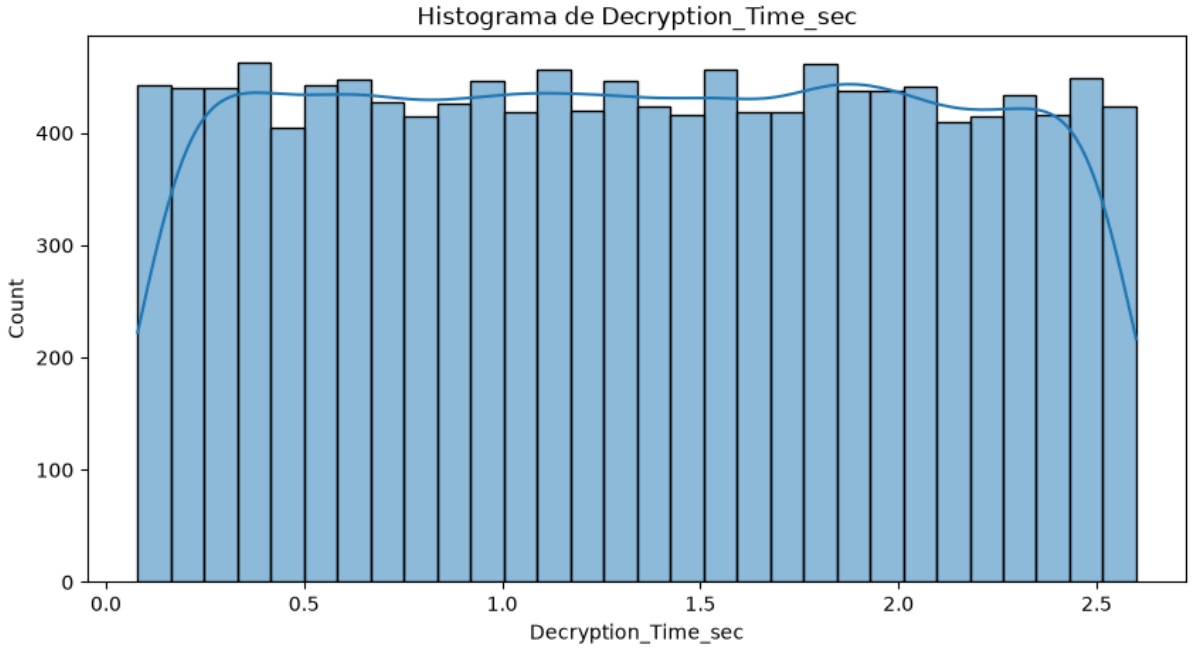

Histograma de Local_Data_Records
Count
0
100
200
300
400
500
0
2000
4000
6000
8000
10000
12000
14000
Local_Data_Records

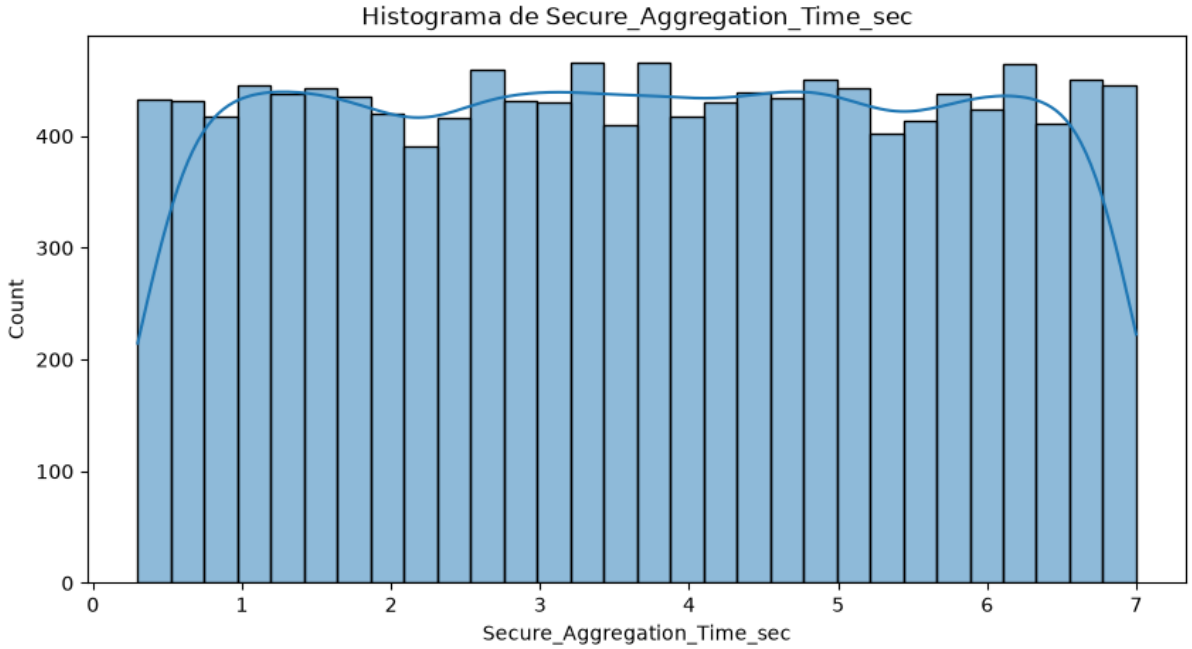

Histograma de Bandwidth_Mbps
Count
Bandwidth_Mbps
0
100
200
300
400
500
0
50
100
150
200
250
300

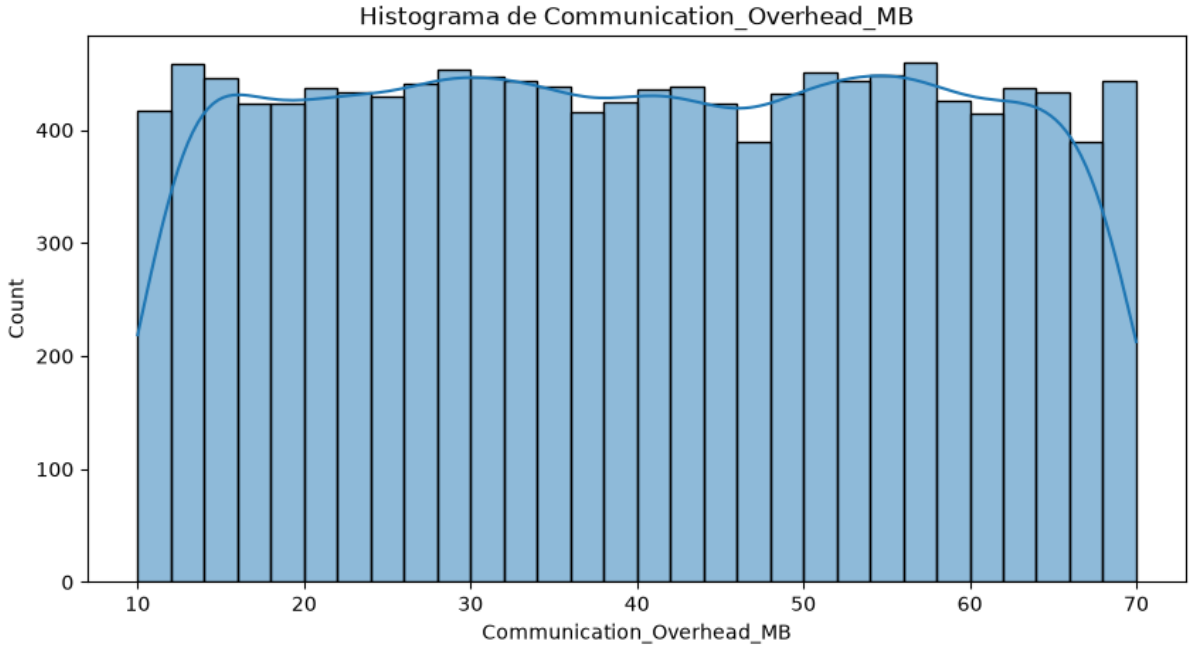

Histograma de Network_Latency_ms
Count
Network_Latency_ms
0
100
200
300
400
500
0
25
50
75
100
125
150
175
200

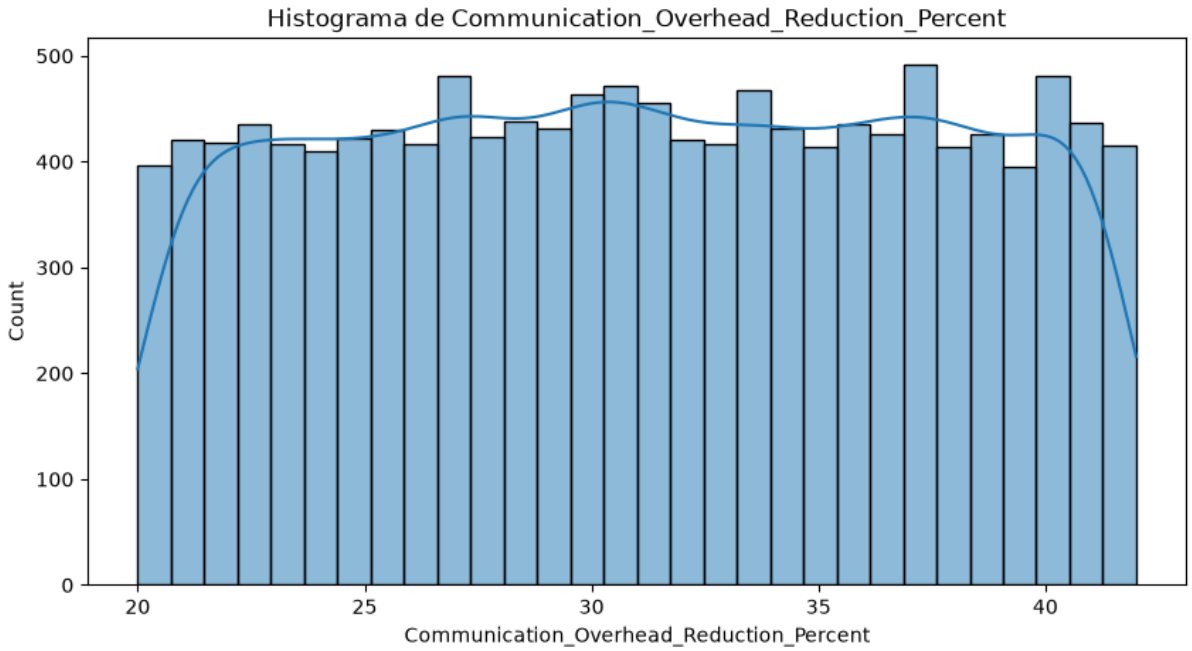

Histograma de Packet_Loss_Rate
Count
Packet_Loss_Rate
0
100
200
300
400
500
0
1
2
3
4
5
6

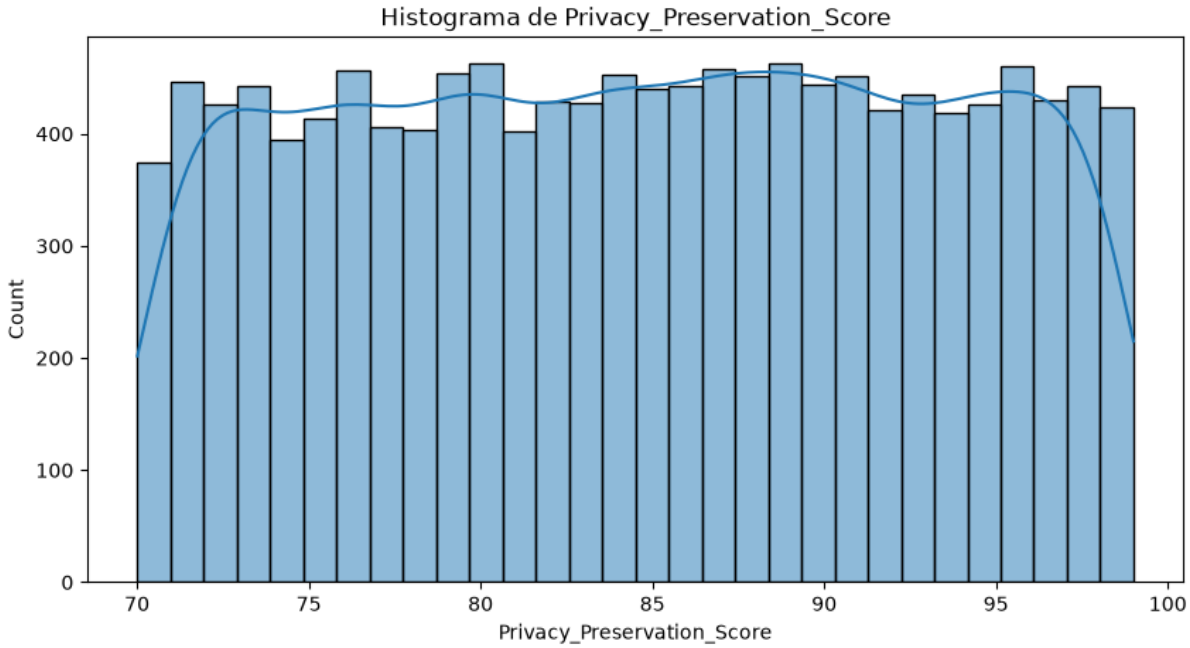

Histograma de Encryption_Time_sec
Count
Encryption_Time_sec
0
100
200
300
400
500
0.0
0.5
1.0
1.5
2.0
2.5
3.0

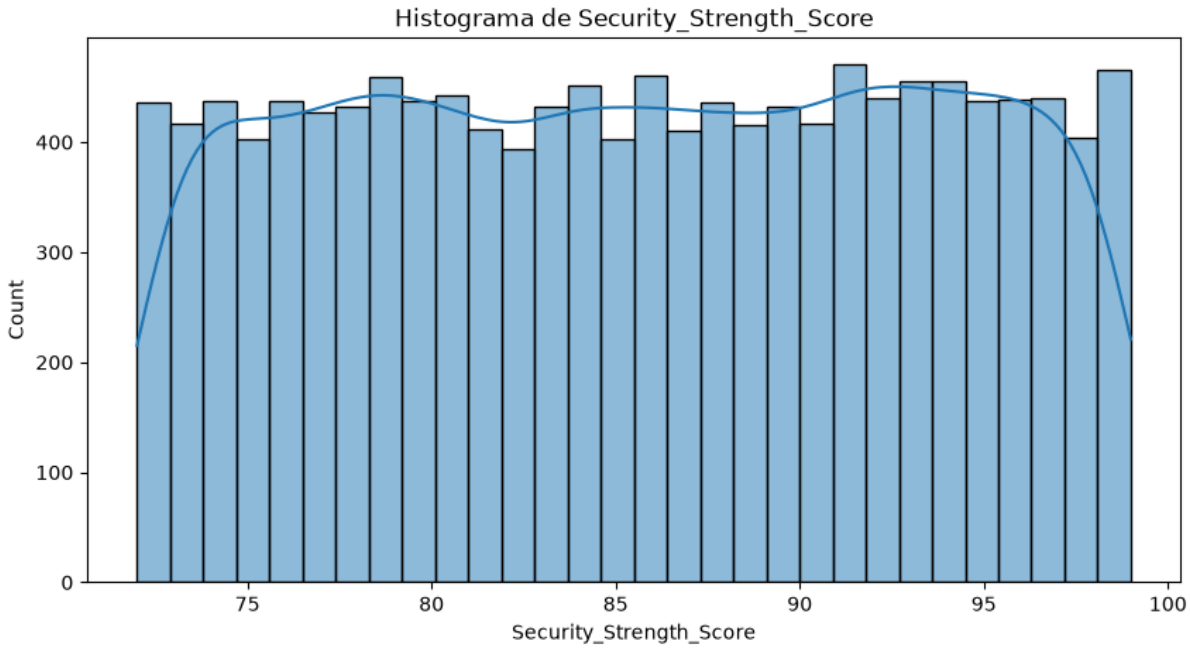

Histograma de Decryption_Time_sec
Count
0
100
200
300
400
0.0
0.5
1.0
1.5
2.0
2.5
Decryption_Time_sec

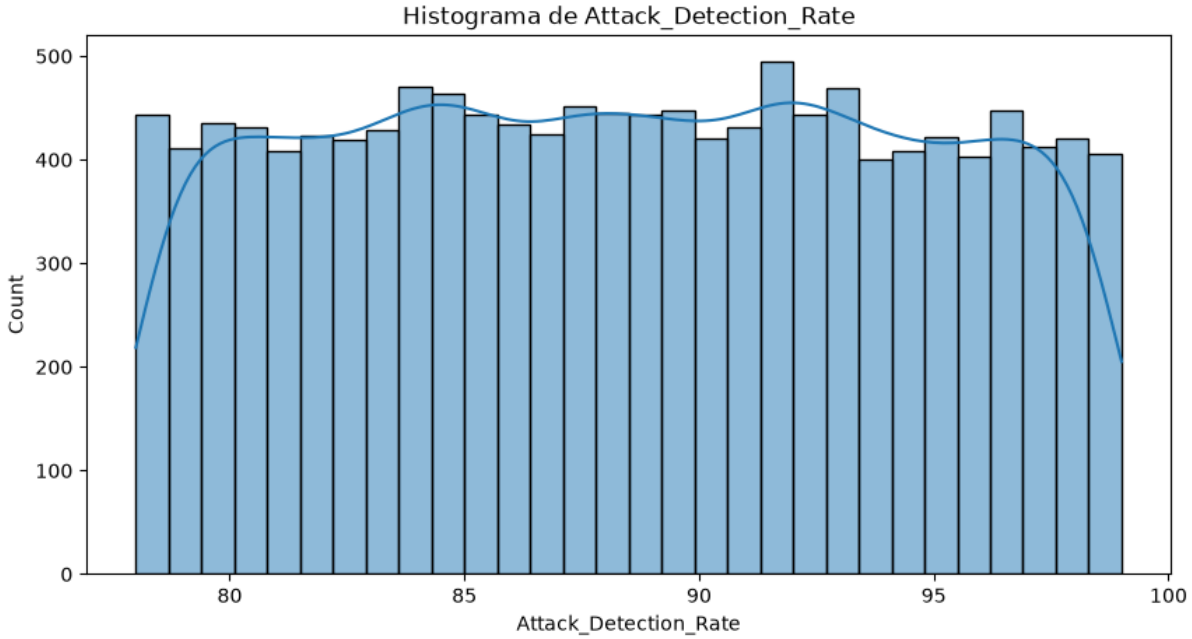

Histograma de Secure_Aggregation_Time_sec
Count
0
100
200
300
400
0
1
2
3
4
5
6
7
Secure_Aggregation_Time_sec

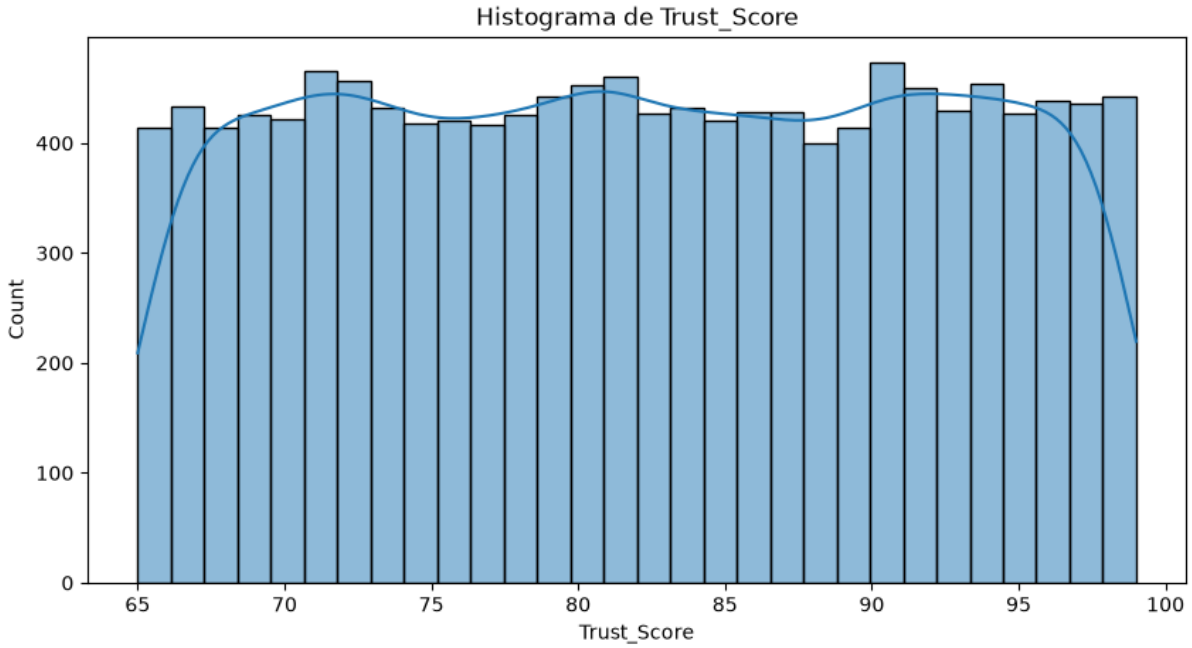

Histograma de Communication_Overhead_MB
Count
0
100
200
300
400
10
20
30
40
50
60
70
Communication_Overhead_MB

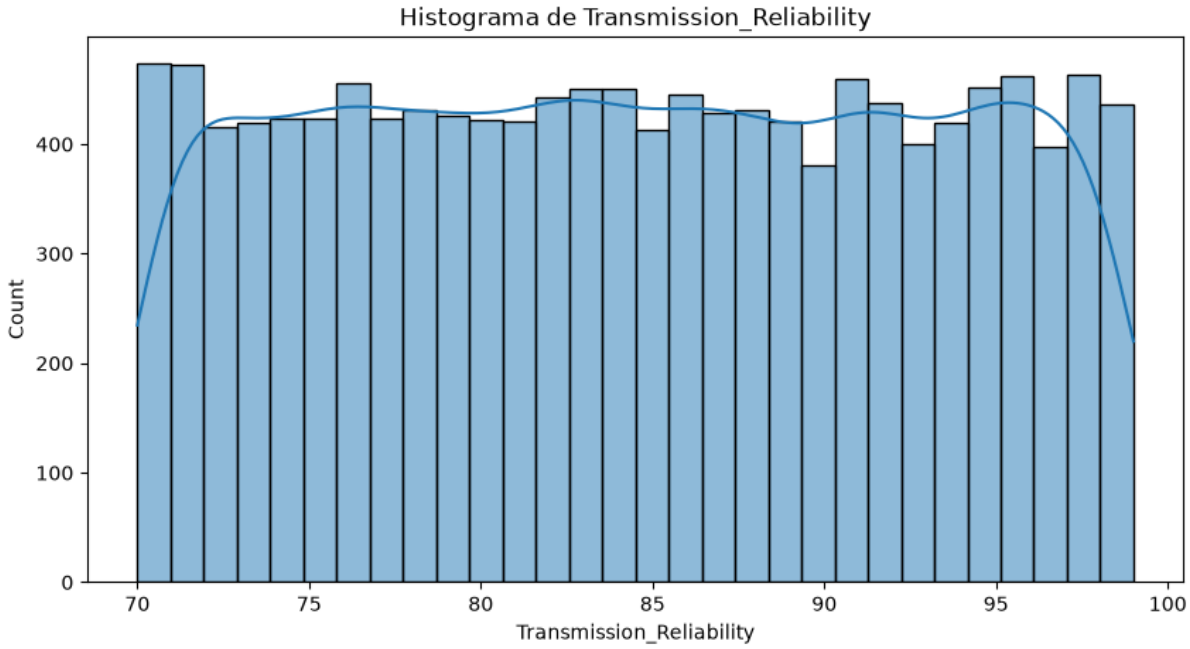

Histograma de Communication_Overhead_Reduction_Percent
Count
0
100
200
300
400
500
20
25
30
35
40
Communication_Overhead_Reduction_Percent


Histograma de Privacy_Preservation_Score
Count
0
100
200
300
400
70
75
80
85
90
95
100
Privacy_Preservation_Score


Histograma de Security_Strength_Score
Count
0
100
200
300
400
75
80
85
90
95
100
Security_Strength_Score

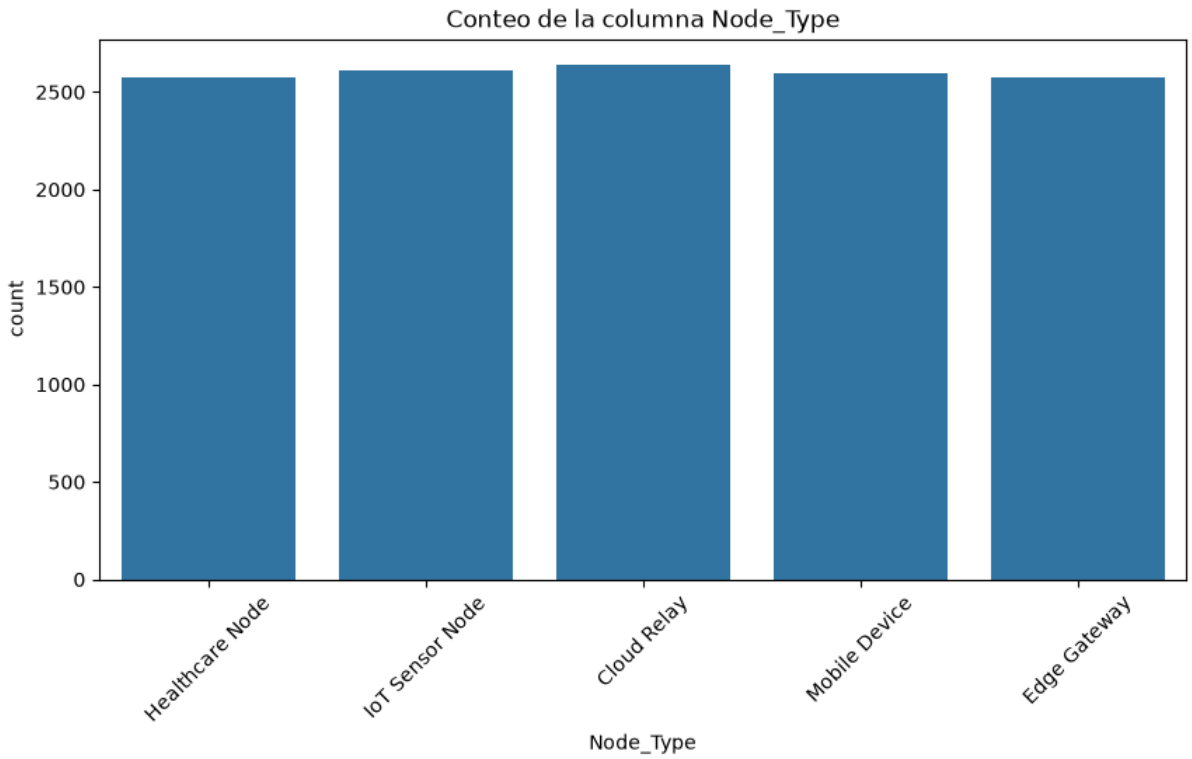

Histograma de Attack_Detection_Rate
Count
Attack_Detection_Rate
0
100
200
300
400
500
80
85
90
95
100

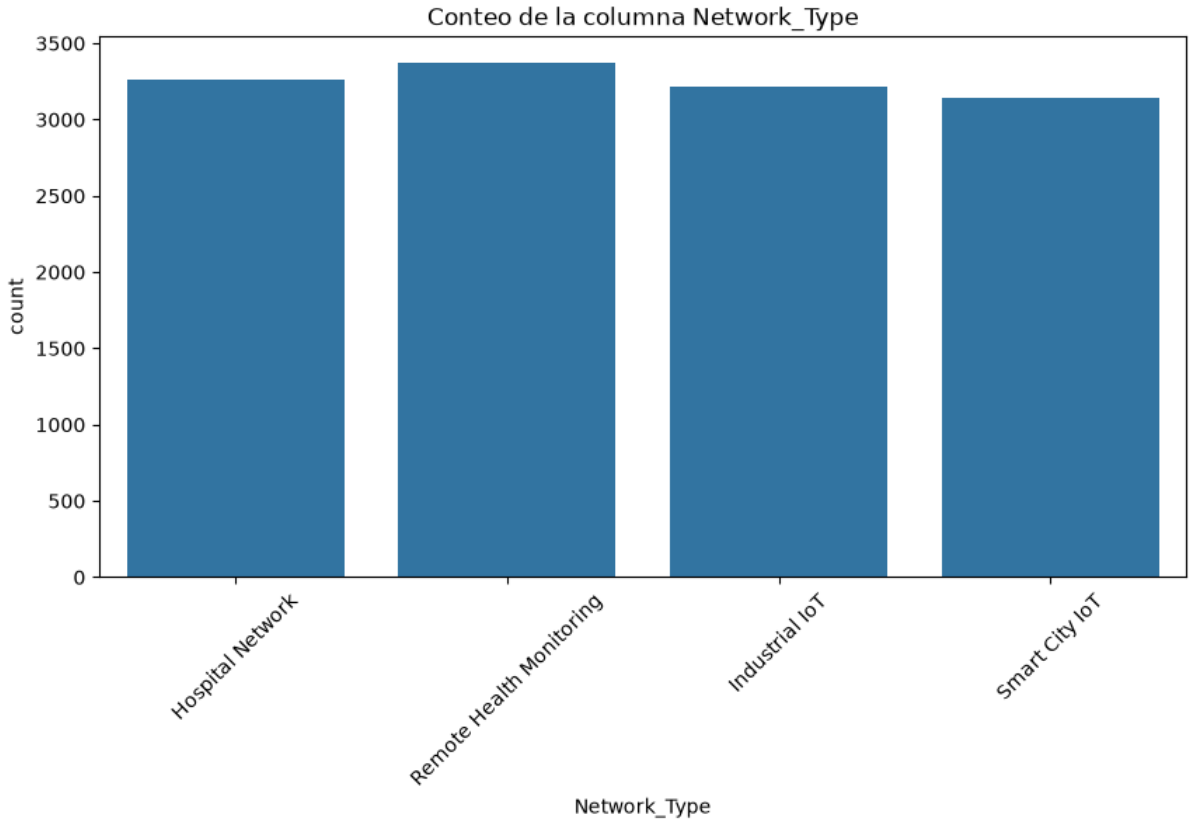

Histograma de Trust_Score
Count
Trust_Score
0
100
200
300
400
65
70
75
80
85
90
95
100

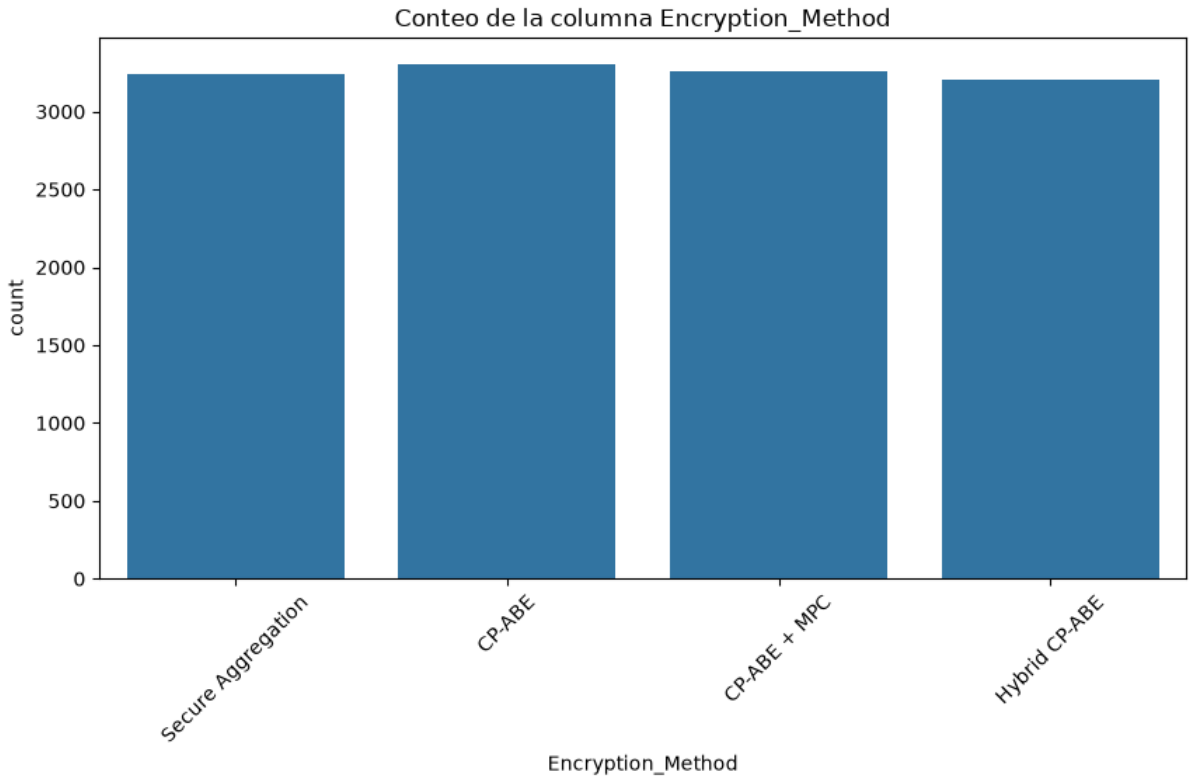


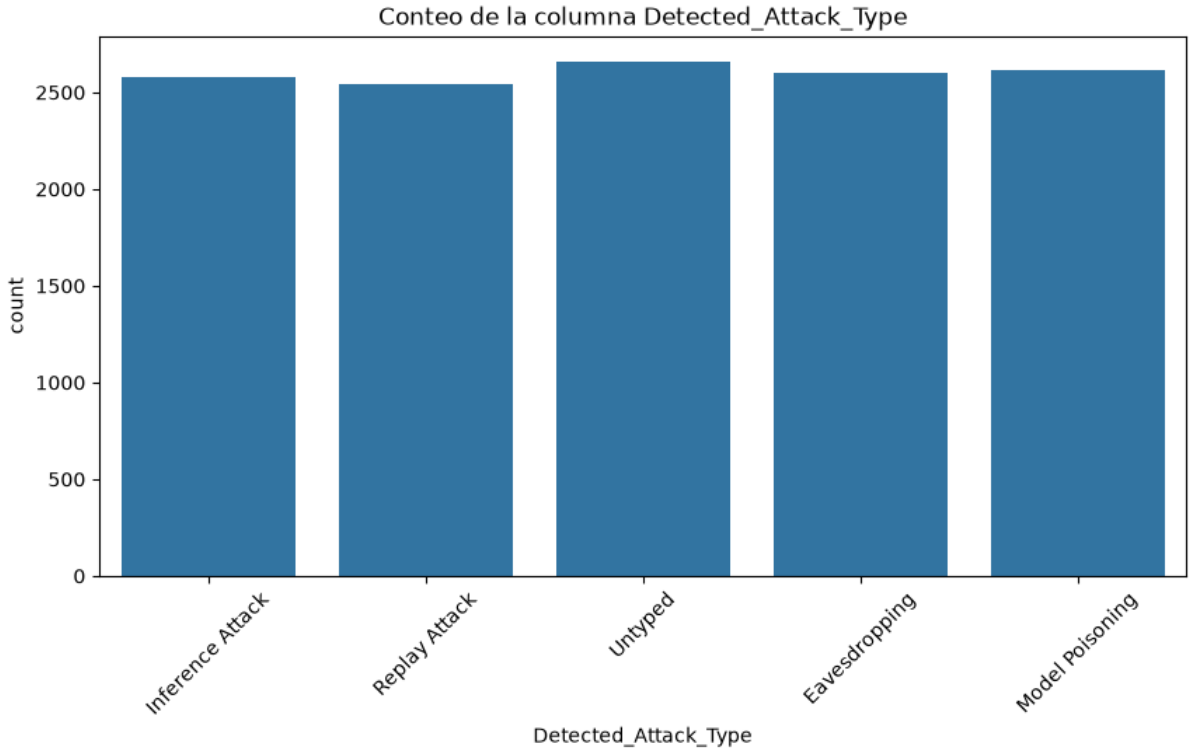
Segun los graficos generados en las variables numericas tanto boxsplot como histograma, nos muestra uniformidad y control por lo que no es posible determinar anomalias en la generacion numerica de univariables.

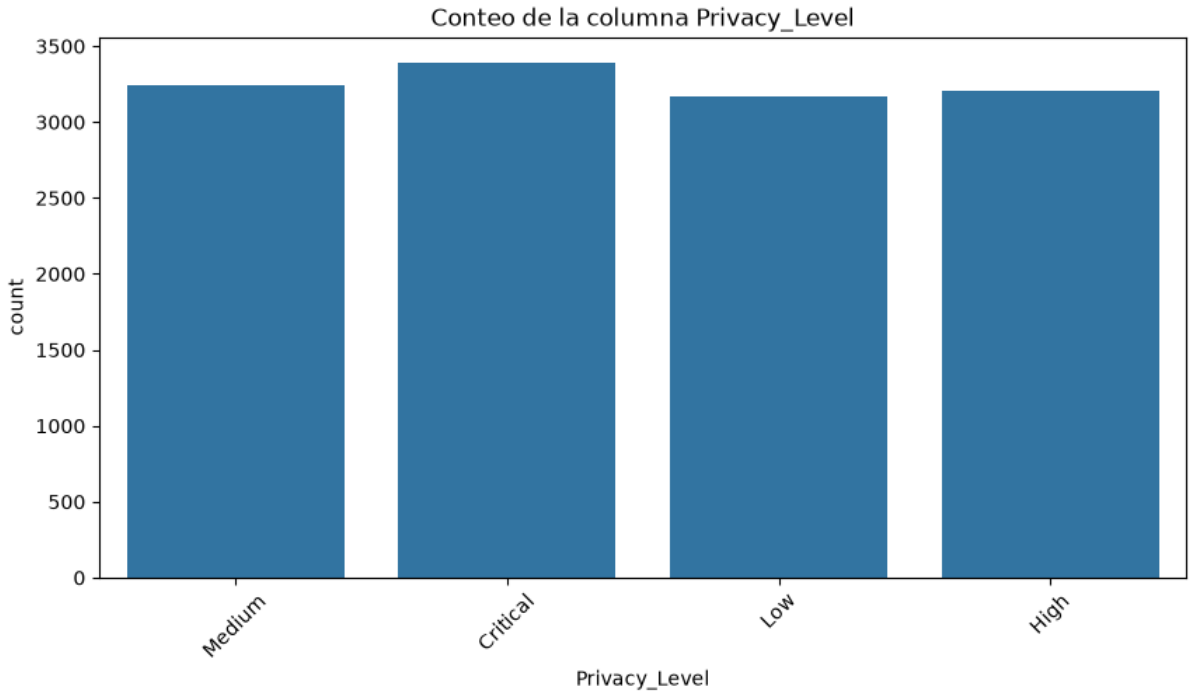
In [32]:
```
for col in df_categoricas.columns:
    plt.figure(figsize=(10, 5))
    sns.countplot(x=df_categoricas[col])
    plt.title(f'Conteo de la columna {col}')
    plt.xticks(rotation=45)
    plt.show()
```

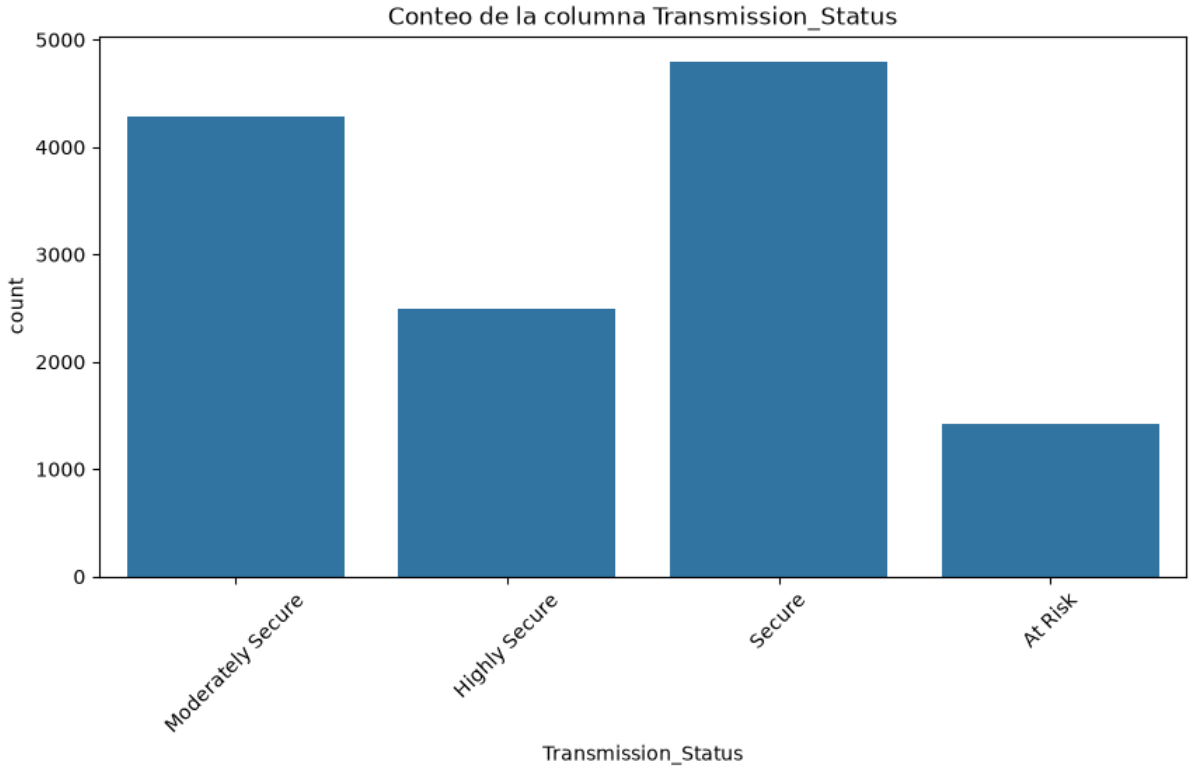



Conteo de la columna Network_Type
count
3500
3000
2500
2000
1500
1000
500
0
Hospital Network
Remote Health Monitoring
Industrial IoT
Smart City IoT
Network_Type


Conteo de la columna Encryption_Method
count
3000
2500
2000
1500
1000
500
0
Secure Aggregation
CP-ABE
CP-ABE + MPC
Hybrid CP-ABE
Encryption_Method


Conteo de la columna Detected_Attack_Type
count
2500
2000
1500
1000
500
0
Inference Attack
Replay Attack
Untyped
Eavesdropping
Model Poisoning
Detected_Attack_Type


Conteo de la columna Privacy_Level
count
3500
3000
2500
2000
1500
1000
500
0
Medium
Critical
Low
High
Privacy_Level



Tambien de forma categorica podemos observar una distribucion balanceada y uniforme, lo cual indica que de esta forma no es posible determinar anormalidades, analizando las variables por si mismas

Analisis Bivariable

In [33]:
```
df_limpio.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 13000 entries, 0 to 12999
Data columns (total 22 columns):
 #   Column                                    Non-Null Count  Dtype
---  ------                                    --------------  -----
 0   Transmission_ID                           13000 non-null  str
 1   Node_Type                                 13000 non-null  str
 2   Network_Type                              13000 non-null  str
 3   Encryption_Method                         13000 non-null  str
 4   Detected_Attack_Type                      13000 non-null  str
 5   Privacy_Level                             13000 non-null  str
 6   Number_of_Distributed_Nodes               13000 non-null  int64
 7   Local_Data_Records                        13000 non-null  int64
 8   Bandwidth_Mbps                            13000 non-null  float64
 9   Network_Latency_ms                        13000 non-null  float64
 10  Packet_Loss_Rate                          13000 non-null  float64
 11  Encryption_Time_sec                       13000 non-null  float64
 12  Decryption_Time_sec                       13000 non-null  float64
 13  Secure_Aggregation_Time_sec               13000 non-null  float64
 14  Communication_Overhead_MB                 13000 non-null  float64
 15  Communication_Overhead_Reduction_Percent  13000 non-null  float64
 16  Privacy_Preservation_Score                13000 non-null  float64
 17  Security_Strength_Score                   13000 non-null  float64
 18  Attack_Detection_Rate                     13000 non-null  float64
 19  Trust_Score                               13000 non-null  float64
 20  Transmission_Reliability                  13000 non-null  float64
 21  Transmission_Status                       13000 non-null  str
dtypes: float64(13), int64(2), str(7)
memory usage: 3.2 MB
```

Para determinar el analisis bivariable es conveniente irnos a los parametros que segun el objetivo y la pregunta del proyecto se plantearon

Objetivos: General: Diseñar un modelo de detección de anomalías por medio de una base de datos que determine variables y límites estándar, para establecer patrones normales y fuera de rango, que servirán para inferir un comportamiento que no es normal en grandes cantidades de tráfico de datos. Especificos: Identificar las variables relevantes en el modelo para predecir con la estructura de datos suficientes. Desarrollar un sistema de filtrado con límites establecidos en rangos de normalidad y anormalidad. Comparar diferentes modelos de machine learning con parámetros de eficiencia. Seleccionar el modelo más productivo para obtener los resultados de anomalía eficientemente. Demostrar los resultados con la tasa de pérdida de datos y la fiabilidad por medio de pruebas de confirmación con el tráfico de datos común, e identificar dichas probables anomalías.

Pregunta de investigación: ¿Qué variables de conexión a la red permiten detectar anomalías que afecten la confiabilidad y la pérdida de paquetes en la

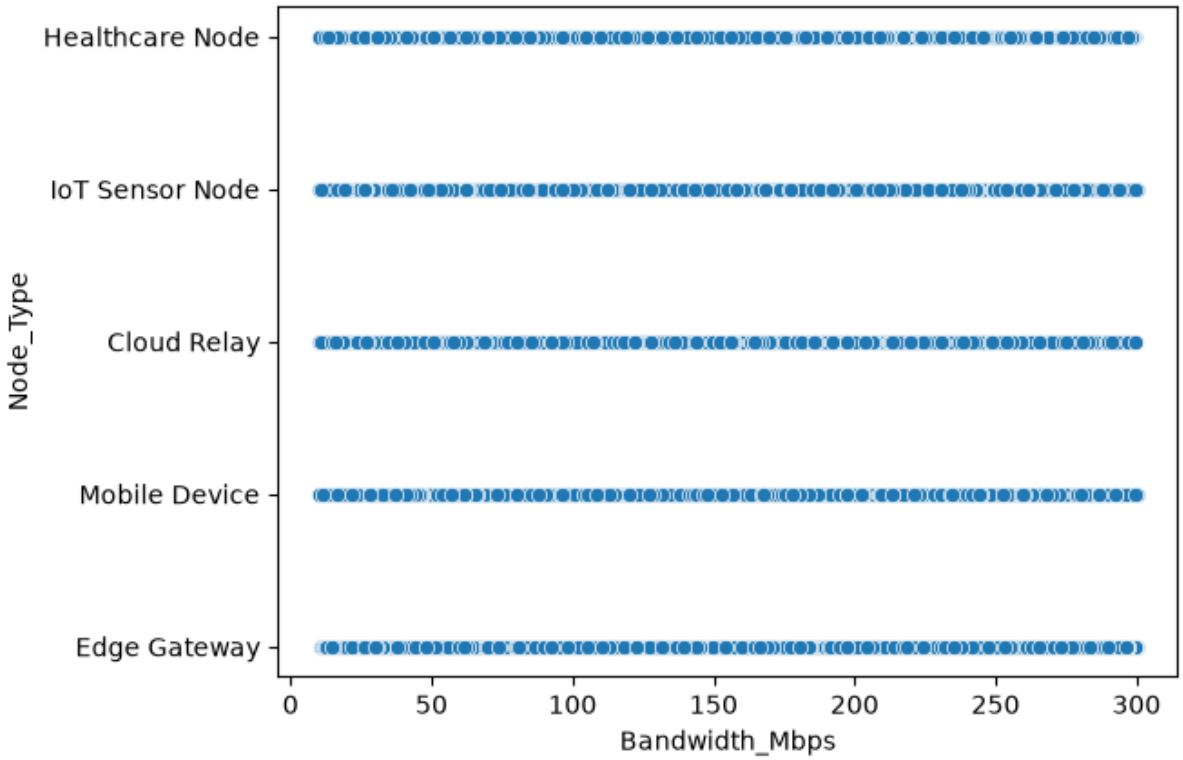
transferencia de datos?

Las combinaciones de variables podrian ser las siguiuentes, considerando tanto numericas como categoricas:

1. Node Type vs Bandwidth
2. Network Type vs Network Latency
3. Bandwidth vs Network Latency
4. Network Latency vs Packet Loss Rate
5. Trust Score vs Packet Loss Rate

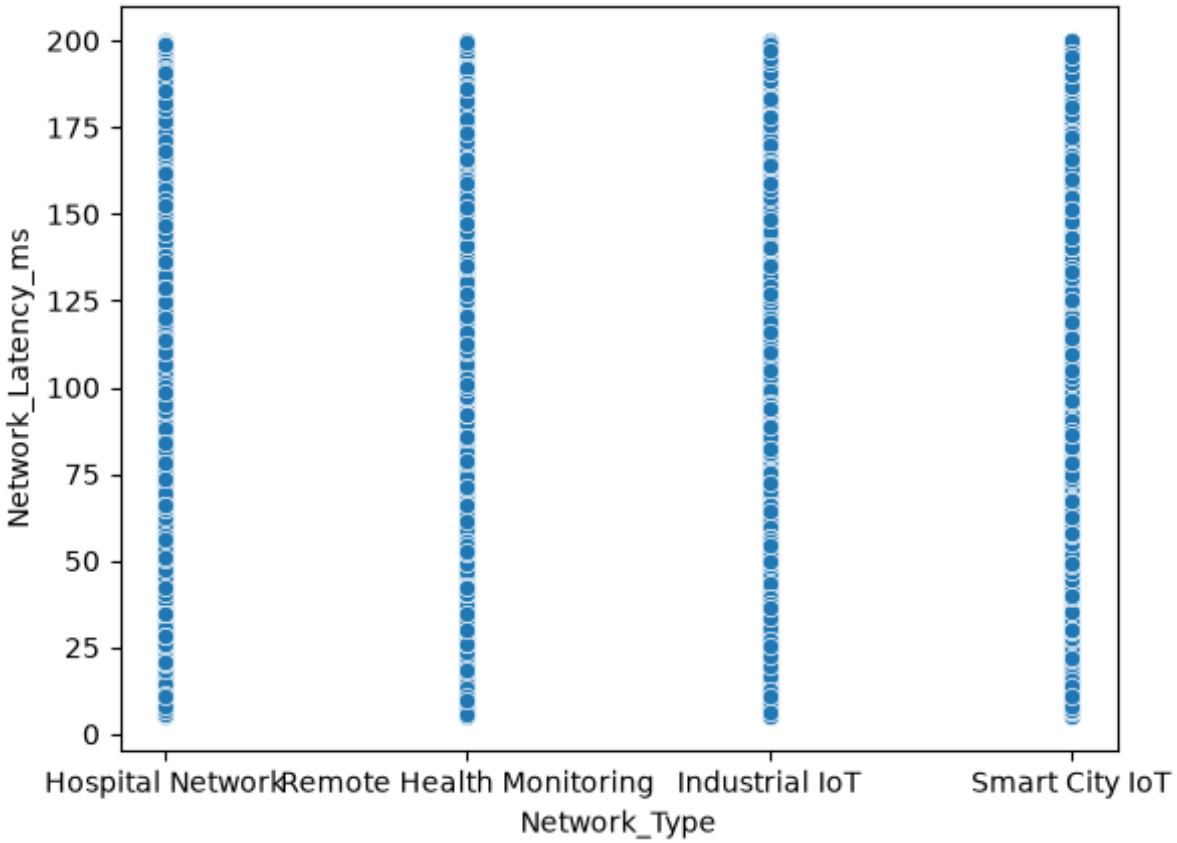
In [34]:
```
sns.scatterplot(y='Node_Type', x='Bandwidth_Mbps', data=df_limpio)
```

Out[34]:
```
<Axes: xlabel='Bandwidth_Mbps', ylabel='Node_Type'>
```



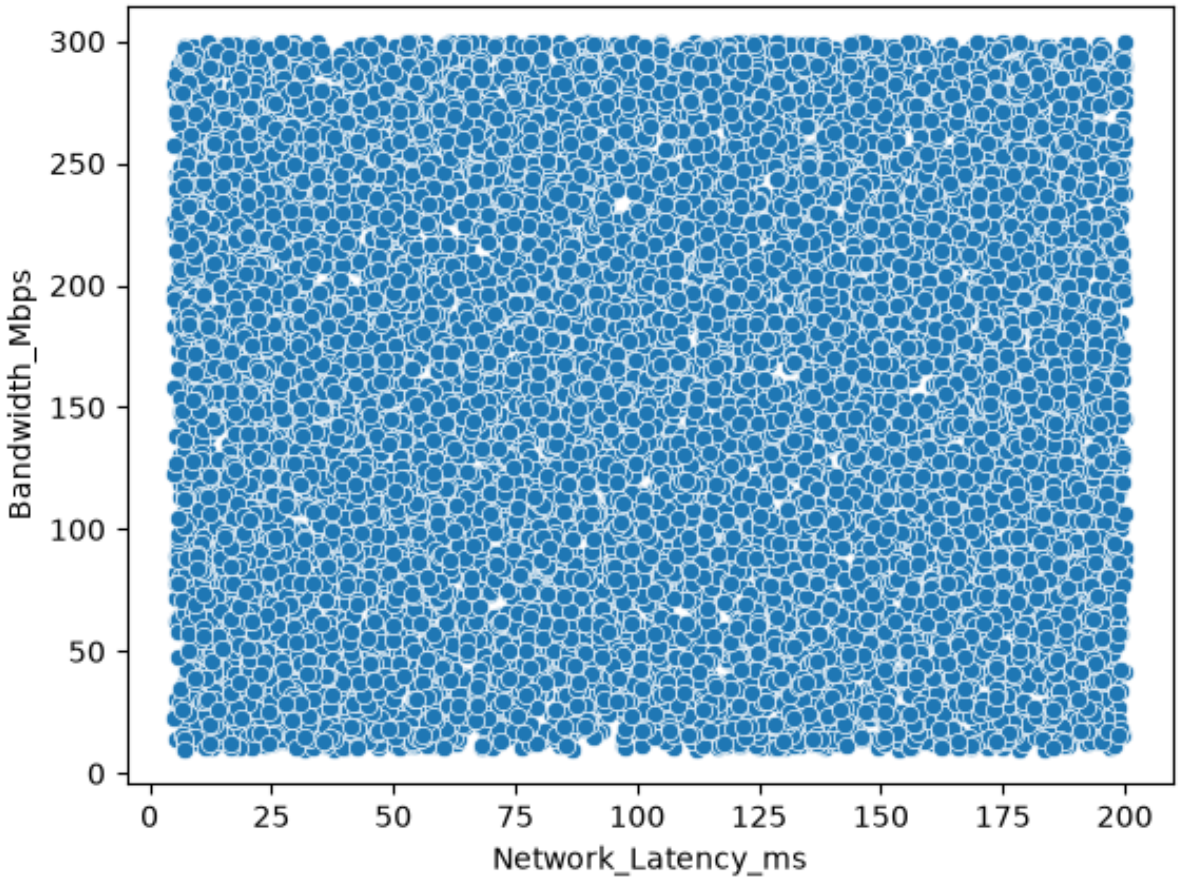
In [35]:
```
sns.scatterplot(x='Network_Type', y='Network_Latency_ms', data=df_limp
```

Out[35]:
```
<Axes: xlabel='Network_Type', ylabel='Network_Latency_ms'>
```



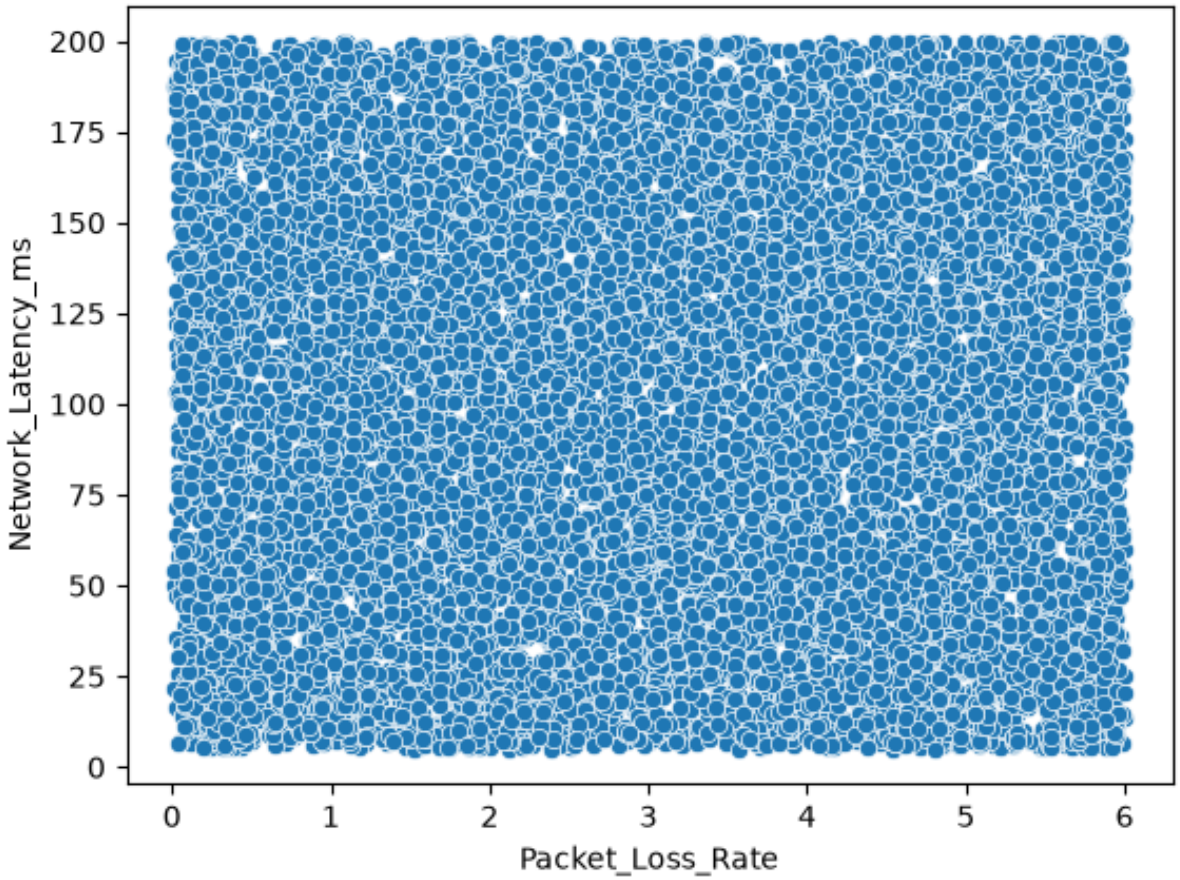
In [36]:
```
sns.scatterplot(x='Network_Latency_ms', y='Bandwidth_Mbps', data=df_li
```

Out[36]:
```
<Axes: xlabel='Network_Latency_ms', ylabel='Bandwidth_Mbps'>
```



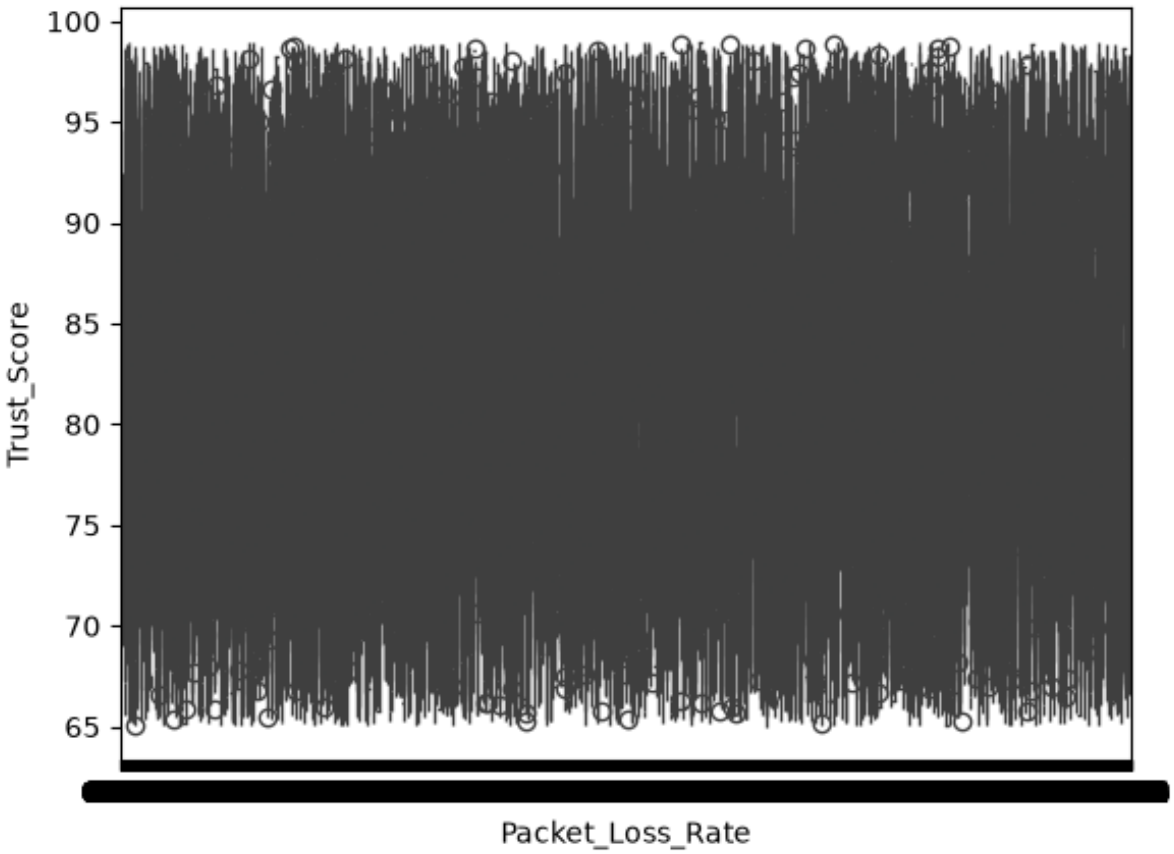
In [37]:
```
sns.scatterplot(x='Packet_Loss_Rate', y='Network_Latency_ms', data=df_
```

Out[37]:
```
<Axes: xlabel='Packet_Loss_Rate', ylabel='Network_Latency_ms'>
```



In [38]:
```
sns.boxplot(x='Packet_Loss_Rate', y='Trust_Score', data=df_limpio)
```

Out[38]:
```
<Axes: xlabel='Packet_Loss_Rate', ylabel='Trust_Score'>
```



Observamos gran dispersion de datos por lo que aun no es posible determinar posibles anormalidades

Lo que sigue es generar en las variables categoricas la transformacion de categoricas a numericas por medio de One Hot Encoding

In [39]:
```
from sklearn.preprocessing import OneHotEncoder
```

In [40]:
```
encoder = OneHotEncoder(sparse_output=False, drop=None)
encoded_categorica = encoder.fit_transform(df_categoricas)
encoded_features = encoder.get_feature_names_out(df_categoricas.column
df_encoded = pd.DataFrame(encoded_categorica, columns=encoded_features

cols_numericas = df.select_dtypes(include=np.number).columns
df_final = pd.concat([df_numericas, df_encoded], axis=1)

print("Columnas procesadas: ", df_final.shape)
```

```
Columnas procesadas:  (13000, 41)
```

In [41]:
```
df_final.info
```

Out[41]:
```
<bound method DataFrame.info of        Number_of_Distributed_Nodes  L
ocal_Data_Records  Bandwidth_Mbps  \
0                                56                1460           63.2
0
```

```
1                              53                9266          187.4
4
2                              13                8113           38.3
2
3                              66                1595          183.3
9
4                               5                2627           95.1
1
...                           ...                 ...
...
12995                          30                6866          230.5
0
12996                          55               10028           48.4
2
12997                          77               12636          208.7
5
12998                          73               11244           37.2
0
12999                           5                1223          109.7
9

       Network_Latency_ms  Packet_Loss_Rate  Encryption_Time_sec  \
0                  157.04             3.585               1.3483
1                   32.20             1.760               1.1258
2                  138.43             2.647               0.4417
3                  184.77             0.540               0.6488
4                    7.75             1.201               2.0918
...                   ...               ...                  ...
12995              151.69             1.646               2.8480
12996              143.34             2.266               0.9617
12997              146.92             5.663               2.2306
12998               18.49             3.021               2.2915
12999               62.73             5.857               1.7014

       Decryption_Time_sec  Secure_Aggregation_Time_sec  \
0                   0.3319                       3.3770
1                   1.2293                       5.5607
2                   1.3278                       0.5304
3                   0.1940                       2.4797
4                   2.0712                       4.3599
...                    ...                          ...
12995               2.5098                       2.4562
12996               0.7081                       0.7377
12997               2.4264                       3.4942
12998               1.7280                       1.6270
12999               2.0941                       5.1617

       Communication_Overhead_MB  Communication_Overhead_Reduction_Pe
rcent  \
0                          30.02
23.14
1                          21.98
```

```
31.31
2                         64.56
25.69
3                         33.32
25.97
4                         65.58
34.32
...                         ...
...
12995                     59.19
26.22
12996                     32.17
21.48
12997                     69.46
32.28
12998                     42.13
36.07
12999                     36.33
20.90

       ...  Detected_Attack_Type_Replay Attack  Detected_Attack_Type_
Untyped  \
0      ...                                0.0
0.0
1      ...                                1.0
0.0
2      ...                                0.0
0.0
3      ...                                1.0
0.0
4      ...                                0.0
1.0
...    ...                                ...
...
12995  ...                                0.0
0.0
12996  ...                                0.0
0.0
12997  ...                                0.0
0.0
12998  ...                                0.0
0.0
12999  ...                                1.0
0.0

       Privacy_Level_Critical  Privacy_Level_High  Privacy_Level_Low
\
0                         0.0                 0.0                0.0
1                         1.0                 0.0                0.0
2                         0.0                 0.0                0.0
3                         1.0                 0.0                0.0
4                         0.0                 0.0                0.0
```

```
...                      ...                 ...                 ...
12995                    0.0                 1.0                 0.0
12996                    0.0                 1.0                 0.0
12997                    0.0                 1.0                 0.0
12998                    1.0                 0.0                 0.0
12999                    0.0                 1.0                 0.0

       Privacy_Level_Medium  Transmission_Status_At Risk  \
0                       1.0                          0.0
1                       0.0                          0.0
2                       1.0                          0.0
3                       0.0                          0.0
4                       1.0                          0.0
...                     ...                          ...
12995                   0.0                          0.0
12996                   0.0                          0.0
12997                   0.0                          1.0
12998                   0.0                          0.0
12999                   0.0                          0.0

       Transmission_Status_Highly Secure  \
0                                    0.0
1                                    1.0
2                                    0.0
3                                    0.0
4                                    1.0
...                                  ...
12995                                0.0
12996                                0.0
12997                                0.0
12998                                0.0
12999                                0.0

       Transmission_Status_Moderately Secure  Transmission_Status_Sec
ure
0                                        1.0
0.0
1                                        0.0
0.0
2                                        1.0
0.0
3                                        0.0
1.0
4                                        0.0
0.0
...                                      ...
...
12995                                    0.0
1.0
12996                                    0.0
1.0
12997                                    0.0
```

```
0.0
12998                                          0.0
1.0
12999                                          1.0
0.0

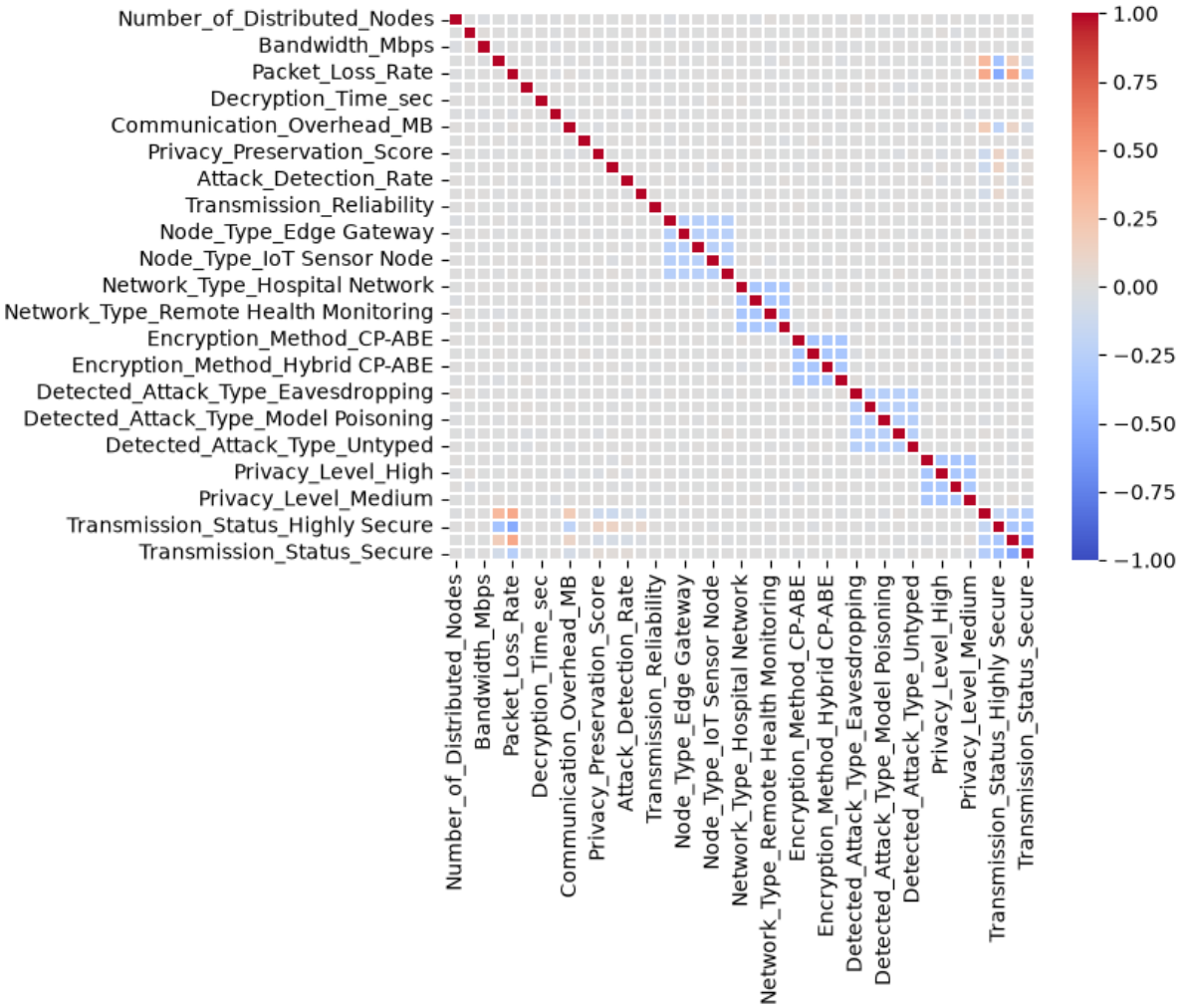
[13000 rows x 41 columns]>
```

In [42]:
```
df_final.columns
```

Out[42]:
```
Index(['Number_of_Distributed_Nodes', 'Local_Data_Records', 'Bandwidt
h_Mbps',
       'Network_Latency_ms', 'Packet_Loss_Rate', 'Encryption_Time_se
c',
       'Decryption_Time_sec', 'Secure_Aggregation_Time_sec',
       'Communication_Overhead_MB', 'Communication_Overhead_Reduction
_Percent',
       'Privacy_Preservation_Score', 'Security_Strength_Score',
       'Attack_Detection_Rate', 'Trust_Score', 'Transmission_Reliabil
ity',
       'Node_Type_Cloud Relay', 'Node_Type_Edge Gateway',
       'Node_Type_Healthcare Node', 'Node_Type_IoT Sensor Node',
       'Node_Type_Mobile Device', 'Network_Type_Hospital Network',
       'Network_Type_Industrial IoT', 'Network_Type_Remote Health Mon
itoring',
       'Network_Type_Smart City IoT', 'Encryption_Method_CP-ABE',
       'Encryption_Method_CP-ABE + MPC', 'Encryption_Method_Hybrid CP
-ABE',
       'Encryption_Method_Secure Aggregation',
       'Detected_Attack_Type_Eavesdropping',
       'Detected_Attack_Type_Inference Attack',
       'Detected_Attack_Type_Model Poisoning',
       'Detected_Attack_Type_Replay Attack', 'Detected_Attack_Type_Un
typed',
       'Privacy_Level_Critical', 'Privacy_Level_High', 'Privacy_Level
_Low',
       'Privacy_Level_Medium', 'Transmission_Status_At Risk',
       'Transmission_Status_Highly Secure',
       'Transmission_Status_Moderately Secure', 'Transmission_Status_
Secure'],
      dtype='str')
```

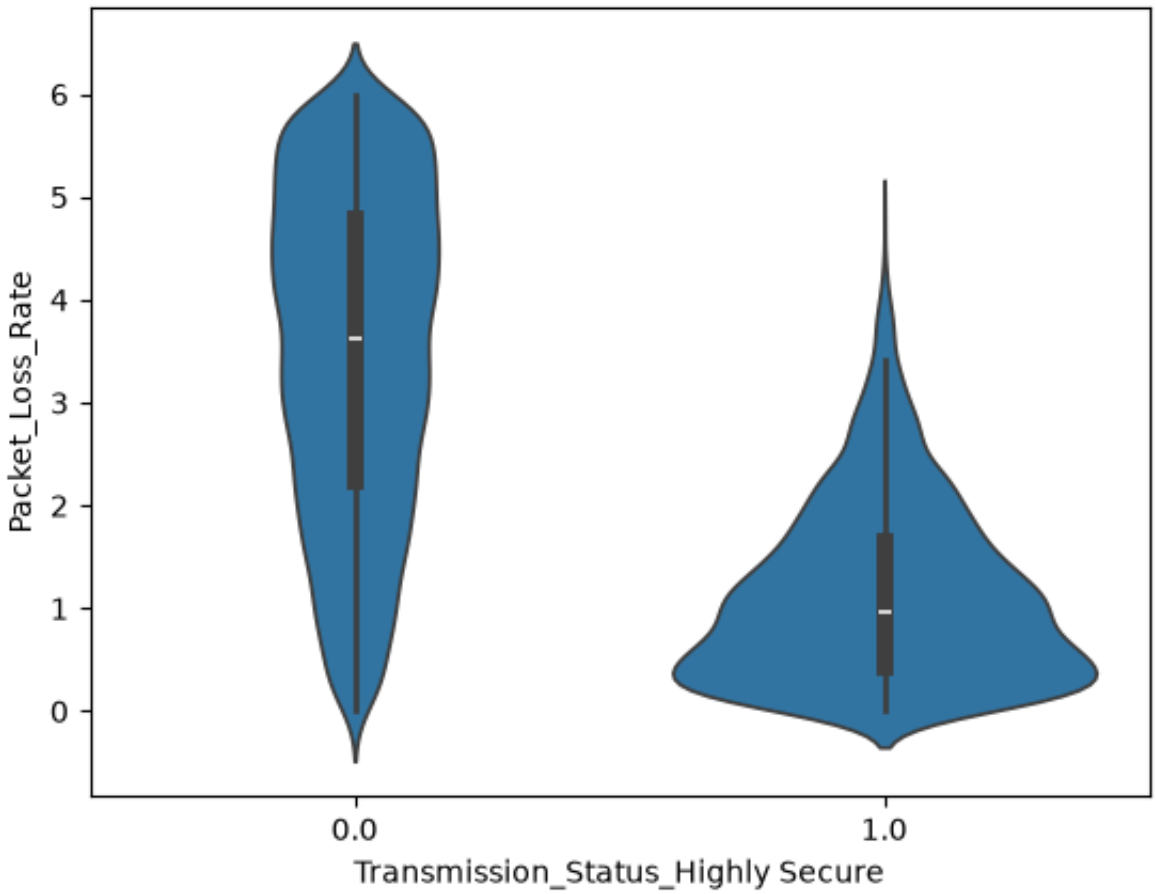
In [43]:
```
sns.heatmap(df_final.corr(), annot=False, cmap='coolwarm', vmax=1, vmi
```

Out[43]:
```
<Axes: >
```



In [44]:
```
sns.violinplot(x='Transmission_Status_Highly Secure', y='Packet_Loss_R
```

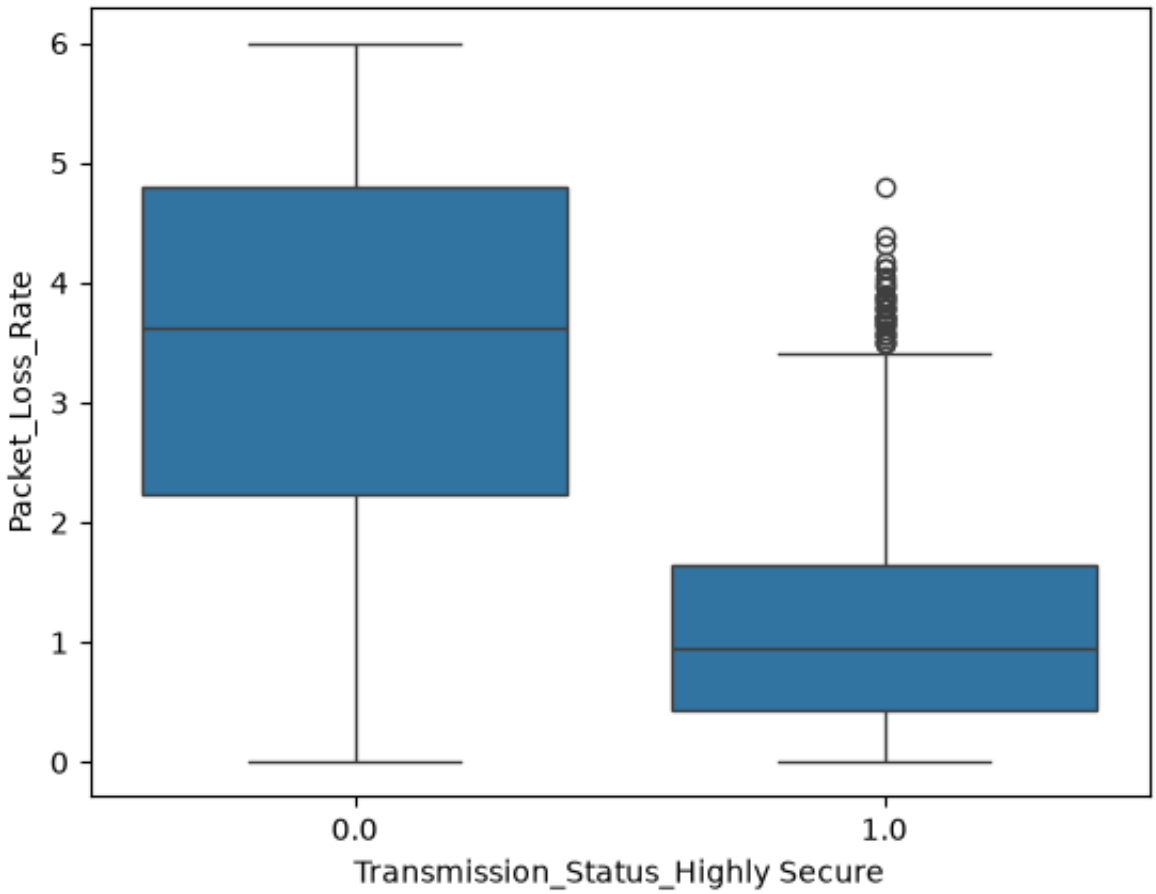
Out[44]:
```
<Axes: xlabel='Transmission_Status_Highly Secure', ylabel='Packet_Los
s_Rate'>
```



En el violinplot se definen dos graficos el 0.0 que indica estatus de seguridad no segura o no altamente segura, el cual podemos encontrar dispersion de datos desde 0 a 6 en perdida de paquetes, la caja se encuentra entre 2 y 5 y la mediana aprox en 3.6, no se observa simetria en los datos, ni tampoco outliers se distribuyen a lo largo de las opciones de la perdida de paquetes. En el 1.0 que indica estatus de transmision altamente segura contamos con alta concentracion de datos entre 0 y 1 ademas que la caja se encuentra entre 0.6 y 1.6 aprox, ademas de un pico alto desde el 3.5 en adelante por lo que es posible decir que cuenta con ciertos outliers

In [45]:
```
sns.boxplot(x='Transmission_Status_Highly Secure', y='Packet_Loss_Rate
```

Out[45]:
```
<Axes: xlabel='Transmission_Status_Highly Secure', ylabel='Packet_Los
s_Rate'>
```



En el boxplot que donde si existe Estatus de transmision tenemos ciertos outliers, se observa cierto rango de asimetria, los datos estan muy agrupados en 1.0, un tasa muy agrupada que va de 0.4 a 1.6, que son transmisiones seguras, la tasa de perdida de paquetes es mas baja donde la mediana se reduce a 0.9 lo cual indica un rendimiento de red mas eficiente, se observan varios outliers cerca de los niveles de los canales no seguros en cuanto a perdidas. En 0.0 que son transmisiones no seguras la mediana de perdida de paquetes es mayor ubicanndose en 3.6 paquetes, se observa alta dispersion por el Q1 y Q3 que va de 2.2 a 4.8, el rango total va de 0 a 6 sin valores atipicos.

Tras el analisis bivariable y de la correlacion con el mapa de calor multivariables, se observa una alta dispersion de datos, no existe una correlacion lineal simple fuerte, y no obedecen a comportamientos aislados por lo que se cree conveniente el uso de modelo de analisis como Iso Forest, modelo no supervisado para busqueda de lo que es normal o anormal

In [46]:
```
from sklearn.ensemble import IsolationForest
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
```

In [47]:
```
scaler = StandardScaler()
```

```
df_final_scaled = scaler.fit_transform(df_final)
```

In [48]:
```
isolation_forest = IsolationForest(contamination=0.05, random_state=42
df_final['Anomaly'] = isolation_forest.fit_predict(df_final_scaled)
```

In [49]:
```
df_final.head()
```

Out[49]:

	Number_of_Distributed_Nodes	Local_Data_Records	Bandwidth_Mbps	Netwo
0	56	1460	63.20	
1	53	9266	187.44	
2	13	8113	38.32	
3	66	1595	183.39	
4	5	2627	95.11	

5 rows × 42 columns

In [50]:
```
df_final.value_counts('Anomaly')
```

Out[50]:
```
Anomaly
 1    12350
-1      650
Name: count, dtype: int64
```

In [51]:
```
df_final['Anomaly_Label'] = df_final['Anomaly'].map({1: 'Normal', -1:
```

In [52]:
```
print(df_final['Anomaly_Label'].value_counts())
```

```
Anomaly_Label
Normal     12350
Anomaly      650
Name: count, dtype: int64
```

In [53]:
```
df_final.head(1000)
```

Out[53]:

	Number_of_Distributed_Nodes	Local_Data_Records	Bandwidth_Mbps	Ne
0	56	1460	63.20	
1	53	9266	187.44	
2	13	8113	38.32	
3	66	1595	183.39	
4	5	2627	95.11	
...	...	...	...	
995	18	993	79.46	
996	78	7202	268.02	
997	82	1599	283.97	
998	42	1436	72.16	
999	15	12968	22.41	

1000 rows × 43 columns

In [54]:
```
df_final['Puntaje_anomalia'] = isolation_forest.decision_function(df_f
```

In [55]:
```
anomalias_puntaje = df_final[df_final['Anomaly'] == -1].sort_values(by
```

In [56]:
```
print(anomalias_puntaje)
```

```
      Number_of_Distributed_Nodes  Local_Data_Records  Bandwidth_Mbps
\
9776                           12               14487           41.05
9563                            8                1526           32.10
3684                           84               14497          142.93
867                            78                5280           50.81
653                            26                1127          262.77
...                           ...                 ...             ...
676                            53                6552          100.05
3943                           27               12747          155.18
2556                           56               14734          171.51
8788                            6                6532          184.36
2079                           66                2215          129.41

      Network_Latency_ms  Packet_Loss_Rate  Encryption_Time_sec  \
9776               13.02             0.765               2.6431
9563                9.54             3.388               2.0838
3684              197.37             5.292               0.4543
867               189.31             5.599               0.7314
653                75.81             5.710               2.7497
...                  ...               ...                  ...
```

```
676               198.91            0.239              0.5544
3943               46.50            0.104              2.1062
2556              175.36            4.691              0.1628
8788              130.39            5.911              1.7605
2079               39.21            2.773              1.6770

      Decryption_Time_sec  Secure_Aggregation_Time_sec  \
9776               0.3858                       2.5866
9563               0.3121                       3.5733
3684               2.3351                       2.3426
867                1.6283                       0.4876
653                0.4451                       1.4880
...                   ...                          ...
676                2.3292                       0.7058
3943               2.3974                       1.5305
2556               2.1596                       4.5754
8788               2.1966                       4.7778
2079               2.0038                       2.6443

      Communication_Overhead_MB  Communication_Overhead_Reduction_Perce
nt  \
9776                      32.30                                    21.
02
9563                      20.21                                    23.
42
3684                      56.10                                    20.
00
867                       65.91                                    31.
51
653                       61.76                                    28.
64
...                         ...
...
676                       63.21                                    35.
73
3943                      66.08                                    21.
58
2556                      18.00                                    33.
58
8788                      27.38                                    22.
61
2079                      18.81                                    22.
08

      ...  Privacy_Level_High  Privacy_Level_Low  Privacy_Level_Medium
\
9776  ...                 0.0                0.0                   0.0
9563  ...                 0.0                0.0                   1.0
3684  ...                 0.0                1.0                   0.0
867   ...                 0.0                0.0                   0.0
653   ...                 0.0                0.0                   0.0
...   ...                 ...                ...                   ...
```

```
676   ...                  0.0                  1.0                     0.0
3943  ...                  0.0                  0.0                     0.0
2556  ...                  0.0                  1.0                     0.0
8788  ...                  0.0                  0.0                     0.0
2079  ...                  0.0                  0.0                     1.0

      Transmission_Status_At Risk  Transmission_Status_Highly Secure  \
9776                          0.0                                1.0
9563                          0.0                                1.0
3684                          1.0                                0.0
867                           1.0                                0.0
653                           1.0                                0.0
...                           ...                                ...
676                           0.0                                0.0
3943                          0.0                                1.0
2556                          0.0                                0.0
8788                          1.0                                0.0
2079                          0.0                                1.0

      Transmission_Status_Moderately Secure  Transmission_Status_Secure
\
9776                                    0.0                         0.0
9563                                    0.0                         0.0
3684                                    0.0                         0.0
867                                     0.0                         0.0
653                                     0.0                         0.0
...                                     ...                         ...
676                                     1.0                         0.0
3943                                    0.0                         0.0
2556                                    1.0                         0.0
8788                                    0.0                         0.0
2079                                    0.0                         0.0

      Anomaly  Anomaly_Label  Puntaje_anomalia
9776       -1        Anomaly         -0.024095
9563       -1        Anomaly         -0.024019
3684       -1        Anomaly         -0.023884
867        -1        Anomaly         -0.023089
653        -1        Anomaly         -0.022484
...       ...            ...               ...
676        -1        Anomaly         -0.000084
3943       -1        Anomaly         -0.000083
2556       -1        Anomaly         -0.000031
8788       -1        Anomaly         -0.000030
2079       -1        Anomaly         -0.000026

[650 rows x 44 columns]
```

In [57]:
```
anomalias_puntaje.head(10)
```

Out[57]:

	Number_of_Distributed_Nodes	Local_Data_Records	Bandwidth_Mbps	N
9776	12	14487	41.05	
9563	8	1526	32.10	
3684	84	14497	142.93	
867	78	5280	50.81	
653	26	1127	262.77	
5491	58	10648	26.82	
2250	77	6121	102.34	
12202	82	716	286.65	
3714	29	14921	298.30	
8227	72	4574	90.79	

10 rows × 44 columns

In [58]:
```
normales_puntaje = df_final[df_final['Anomaly_Label'] == 'Normal'].sor
```

In [59]:
```
normales_puntaje.head(10)
```

Out[59]:

	Number_of_Distributed_Nodes	Local_Data_Records	Bandwidth_Mbps	N
4410	56	2059	268.44	
8980	73	7427	233.70	
6355	79	13905	272.12	
778	31	697	282.05	
4633	9	8435	19.14	
4630	62	12934	11.98	
435	76	10384	247.99	
7017	26	6003	275.56	
6945	24	8722	291.81	
5420	36	8409	98.05	

10 rows × 44 columns

In [60]:
```
comparacion = (
```

```
    df_final
    .groupby('Anomaly_Label')[df_numericas.columns]
    .agg(['mean', 'median', 'std'])
)

print(
    comparacion.loc[
        :,
        ['Network_Latency_ms', 'Bandwidth_Mbps', 'Packet_Loss_Rate', '
    ]
)
```

```
              Network_Latency_ms                      Bandwidth_Mbps
\
                            mean    median        std           mean    m
edian
Anomaly_Label
Anomaly                108.229323  120.040  65.985849     151.065785  15
0.680
Normal                 102.525708  103.595  55.710380     155.393925  15
4.605

                         Packet_Loss_Rate                  Trust_Score
\
                      std             mean median       std        mean
Anomaly_Label
Anomaly        93.067507         3.436268  4.114  2.090642   80.469708
Normal         83.292277         2.996090  3.008  1.711773   82.154409

                               Decryption_Time_sec
               median        std                mean    median       std
Anomaly_Label
Anomaly        78.885  11.204902            1.355858  1.34850  0.847094
Normal         82.060   9.724976            1.334079  1.33255  0.721160
```

In [61]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [62]:
```
y_target = (df_final['Anomaly'] == -1).astype(int)
```

In [63]:
```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(df_final_scaled, y_target)
```

Out[63]:

▾ RandomForestClassifier ⓘ ?

▸ Parameters

▸ Fitted attributes

In [64]:
```
feature_cols = [
    col for col in df_final.columns
    if col not in {'Anomaly', 'Anomaly_Label', 'Puntaje_anomalia'}
```

```
]

importances = pd.Series(
    rf_model.feature_importances_,
    index=feature_cols
).sort_values(ascending=False)

print(importances)
```

```
Privacy_Preservation_Score                  0.062657
Attack_Detection_Rate                       0.062642
Decryption_Time_sec                         0.062241
Network_Latency_ms                          0.059563
Communication_Overhead_MB                   0.059064
Secure_Aggregation_Time_sec                 0.058002
Packet_Loss_Rate                            0.057489
Trust_Score                                 0.056653
Local_Data_Records                          0.054134
Security_Strength_Score                     0.050127
Transmission_Reliability                    0.049465
Encryption_Time_sec                         0.047350
Bandwidth_Mbps                              0.047026
Communication_Overhead_Reduction_Percent    0.044777
Number_of_Distributed_Nodes                 0.039273
Transmission_Status_At Risk                 0.031078
Node_Type_Mobile Device                     0.018263
Network_Type_Industrial IoT                 0.010482
Detected_Attack_Type_Model Poisoning        0.009359
Transmission_Status_Secure                  0.007767
Detected_Attack_Type_Replay Attack          0.007732
Transmission_Status_Highly Secure           0.007588
Transmission_Status_Moderately Secure       0.007567
Encryption_Method_CP-ABE                    0.007437
Network_Type_Smart City IoT                 0.006069
Privacy_Level_Low                           0.005759
Privacy_Level_Critical                      0.005499
Encryption_Method_Secure Aggregation        0.005298
Encryption_Method_CP-ABE + MPC              0.005280
Privacy_Level_High                          0.005214
Encryption_Method_Hybrid CP-ABE             0.004972
Detected_Attack_Type_Eavesdropping          0.004935
Network_Type_Hospital Network               0.004769
Detected_Attack_Type_Untyped                0.004720
Node_Type_Edge Gateway                      0.004656
Network_Type_Remote Health Monitoring       0.004643
Privacy_Level_Medium                        0.004427
Node_Type_IoT Sensor Node                   0.004385
Detected_Attack_Type_Inference Attack       0.004016
Node_Type_Healthcare Node                   0.003935
Node_Type_Cloud Relay                       0.003688
dtype: float64
```

In [65]:
```
comparacion = (
```

```
    df_final
    .groupby('Anomaly_Label')[df_numericas.columns]
    .agg(['mean', 'median', 'std'])
)

print(
    comparacion.loc[
        :,
        ['Privacy_Preservation_Score',
'Decryption_Time_sec',
'Network_Latency_ms',
'Communication_Overhead_MB',
'Secure_Aggregation_Time_sec',
'Packet_Loss_Rate',
'Trust_Score',
'Local_Data_Records',
'Security_Strength_Score']
    ]
)
```

```
              Privacy_Preservation_Score                  Decryption_Ti
me_sec  \
                                    mean median       std
mean
Anomaly_Label
Anomaly                          83.502338  81.42  9.869078           1.
355858
Normal                           84.692362  84.80  8.213273           1.
334079

                                  Network_Latency_ms
\
                median       std               mean   median        std
Anomaly_Label
Anomaly        1.34850  0.847094         108.229323  120.040  65.985849
Normal         1.33255  0.721160         102.525708  103.595  55.710380

              Communication_Overhead_MB  ... Packet_Loss_Rate Trust_Sco
re  \
                                   mean  ...              std        me
an
Anomaly_Label                           ...
Anomaly                         42.150369  ...         2.090642   80.4697
08
Normal                          39.779386  ...         1.711773   82.1544
09

                                  Local_Data_Records
\
               median        std               mean  median          st
d
Anomaly_Label
Anomaly        78.885  11.204902        7852.293846  7131.5  4840.44774
3
Normal         82.060   9.724976        7769.504534  7742.0  4134.59132
3

              Security_Strength_Score
                                   mean  median       std
Anomaly_Label
Anomaly                      85.463277  84.925  8.760699
Normal                       85.585744  85.690  7.751285

[2 rows x 27 columns]
```

Conclusion: Dentro de lo que es viable decir del modelo desarrollado encontramos con ciertas sorpresas sobre cuales son las variables que determinan en mayor medida la distribucion de peso hacia decidir si alguna transmision de datos es o no anomalia, enfocadas mayormente en eficiencia y velocidad mas aun que precision, de las variables que mayormente se pensaba abria peso en la decision de si es o no anomalia se encontraban la confiabilidad y

la perdida de paquetes, aunque se encontro con mayor peso variables como Preservacion de privacidad, detecccion de ataques, tiempo descifrado y latencia en conexion como las primeras cuatro variables que determinan anormalidad. Por lo tanto tambien fue observable que realmente el hecho de escoger un modelo no supervisado como lo fue Isolation Forest genero la formula adecuada de combinaciones que determinan normalidad ya que de forma individual tanto cruzada o multivariable no fue posible determinar mas que balance, poca correlacion y uniformidad entre variables, asi que la forma en que podemos encontrar anormalidades con mayor eficiencia es por medio de analisis en tiempo real con modelos no supervisados que crucen las variables en su totalidad, para determinar la discriminacion de una posible anormalidad en el trafico de datos.